Lecture 4

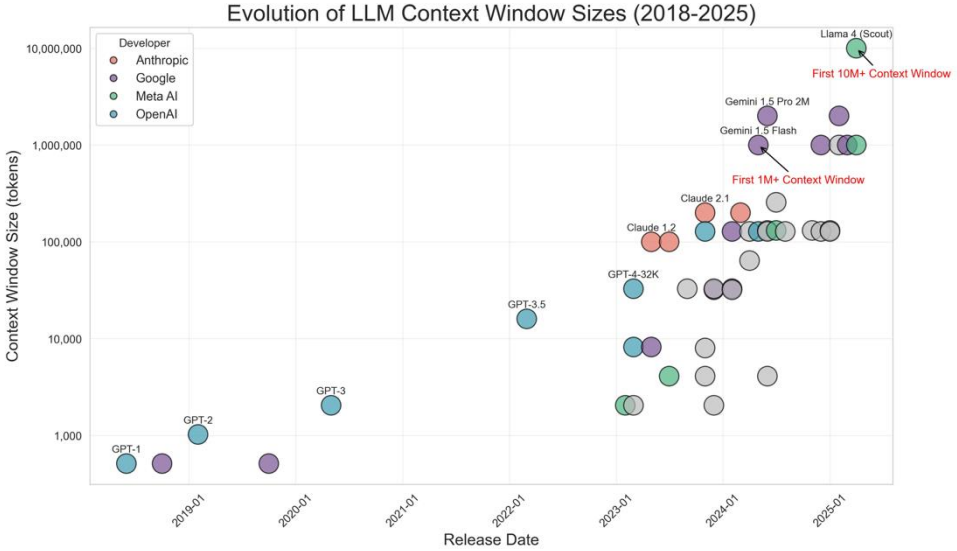
Attention Alternatives and Mixtures of Experts

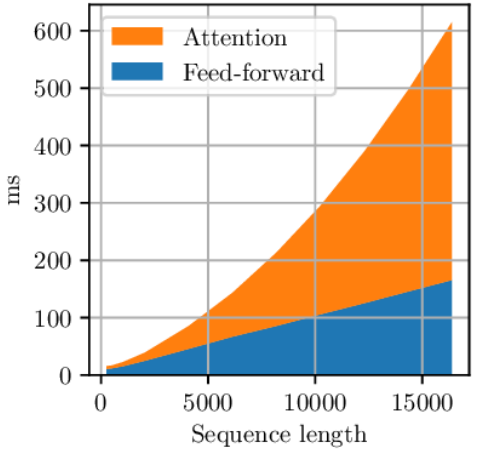
CS336

Tatsu H

Attention alternatives

Cost of attention rises with large context sizes… how do we control those costs?



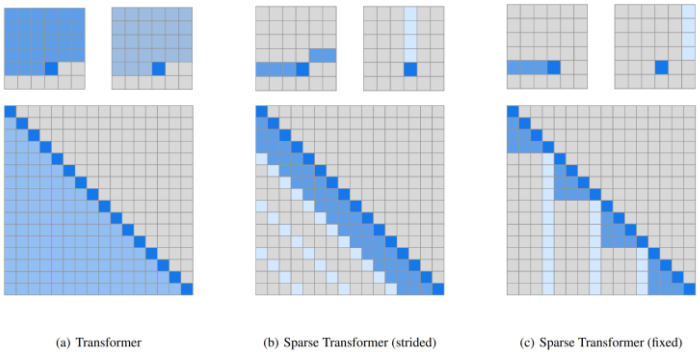


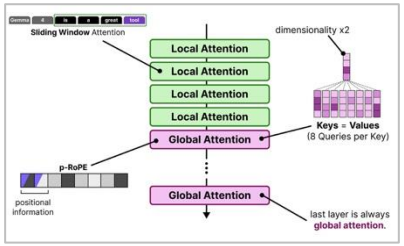
#TODO：这里说的attention和feed-forward所占比例指的是什么？推理的耗时吗？如何理解这两者所占比例随着sequence length的变化？

https://www.meibel.ai/post/understanding-the-impact-of-increasing-llm-context-windows

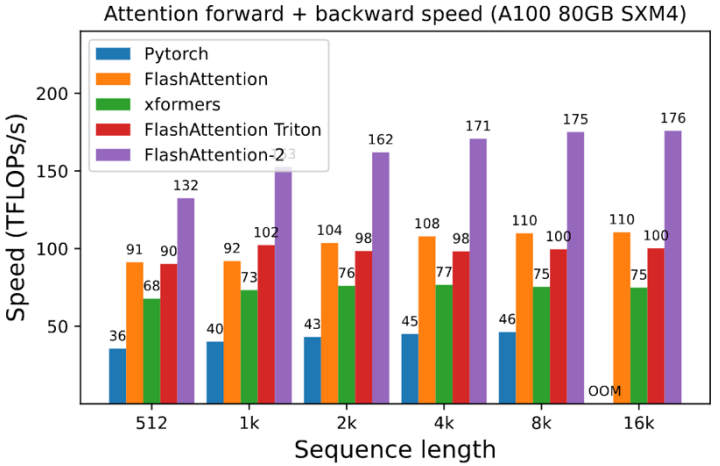
The ‘basic’ toolkit

#TODO：flash attention不改变复杂度，
只改变了所谓的constant factors , which is really matters，这要如何理解?





Combine local + global attention



Systems engineering

But what if we want more radical and potentially large gains?

Linear attention

一般的attention的复杂度（前向）是O(L^2d)的，是和sequence 长度的二次方正比。
我们想要的是一种可以有着linear sequence length 复杂度的attention

Consider the usual attention operation.. $Q \in R^{n \times d_k}, K \in R^{n \times d_k}, V \in R^{n \times d_v}$

序列长度是n，model_dim是dk

$$Attn(Q, K, V) = \rho(QK^\top)V$$

This is quadratic due to $QK^\top$ $(n^2 d_k)$. Can we do better (when ρ is the identity)?

实际看来去除softmax不会造成stability issue，
甚至因为本身instability就是由softmax造成的，
去除后反而增加了稳定性

#TODO：如果不做softmax，那么就被称之为linear attn？这个是linear attn的定义吗？

$$(QK^\top)V = Q(K^\top V)$$

dk和dv一般比n小很多，所以这样转换后flops下降了很多

This is very silly, but surprisingly important.. We get from $n^2 d_k + n^2 d_v$ to $2nd_v d_k$

这里flops省略了常数项2

[Shen et al 2018, Katharopoulos 2020 for kernel version. Also connections to things like fast weight programmers]

Recurrent form of linear attention

Recall that in purely linear attention, we consider the reordering

$$(QK^\top)V = Q(K^\top V)$$

This is linear time (great) but the even nicer thing is that this looks like a RNN.

#TODO：这里具体的推导，为啥这个形式等价于上面的式子。
这里的yt就是第t位置的输出的token吗?
yt的形状是什么?
以及像RNN的好处是什么？duality指的是什么?

$$S_t = S_{t-1} + k_t v_t^\top \quad \text{and} \quad y_t = q_t^\top S_t$$

yt就是输出

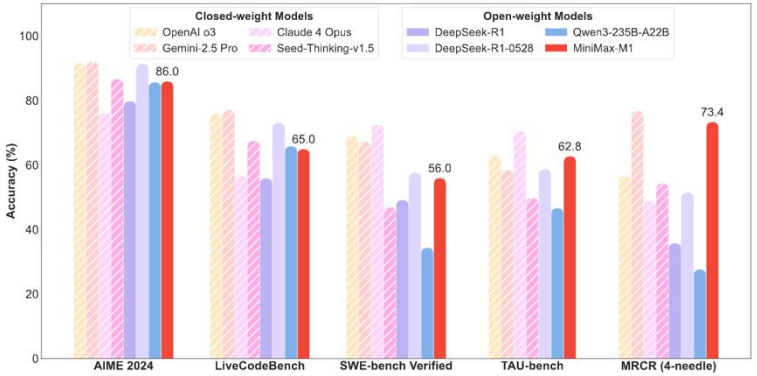
This ‘duality’ allows us to train efficiently (using the parallel, quadratic form) and inference efficiently (using the serial, linear form)

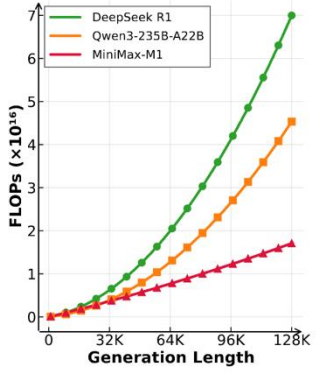
#TODO：和GQA相比，这里的linear attn的目的是啥？GQA我理解是多个query共享一个K,V，降低了KV cache对显存的消耗，同时降低了读取KV cache的大小，缓解memory-bound；那么这里的linear attn的目的，能否理解为当系统已经是compute-bound的时候，通过降低前向所需要的flops从而缓解compute-bound?

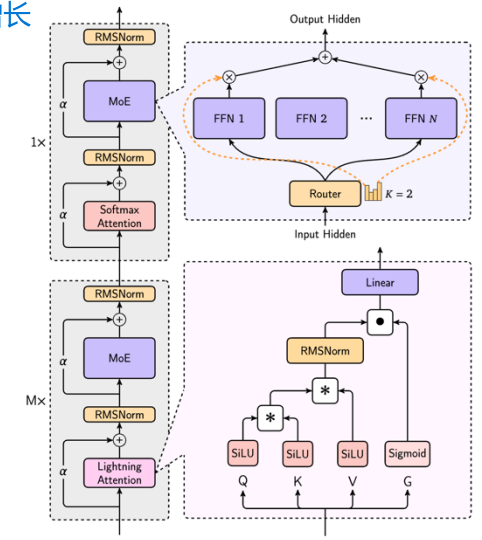
(Note: if one weights S_{t-1} by γ, you get a RetNet)

Minimax M1

linear attn：flops一次方增长







Arch.	BBH ↑	DROP ↑	MMLU ↑	CMMLU ↑	MATH ↑	GSM8k ↑	ARC-C ↑	WG ↑
Softmax	28.2	27.4	49.3	**47.3**	4.6	**18.8**	46.4	65.6
Hybrid-lightning	**32.2**	**29.0**	**49.5**	46.0	**6.8**	18.5	**47.4**	**67.5**

Minimax M1 (and minimax-text-01) use a 7-to-1 hybrid (7 linear, 1 full) linear attention.

Performance (generally) seems strong, and they show linear scaling in context length

From linear attention to Mamba-2

#TODO：Mamba-2的详细解释

Let's generalize linear attention a little bit and add per-position weights..

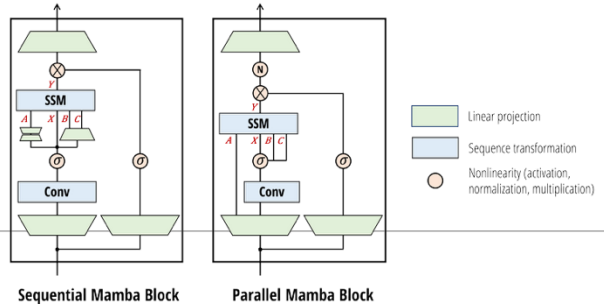
$$S_t = S_{t-1} + k_t v_t^\top \quad \text{and} \quad y_t = q_t^\top S_t \text{ (Linear attention)}$$

$$S_t = \gamma_t S_{t-1} + k_t v_t^\top \quad \text{and} \quad y_t = q_t^\top S_t + v_t^\top D \text{ with } \gamma_t = f(x_t) \text{ (Mamba-2)}$$

这个思路也是和LSTM类似的，用一个系数来控制什么时候passing info forward，什么时候不passing（γt=0）

There is a lot more words to justify this (go read the mamba 2 paper) but the mechanics is that we can make linear attention more expressive via gating (gating is good!)

This continues to have duality properties (compute γ in parallel, apply duality)



Nemotron 3

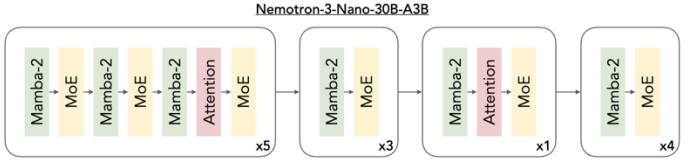


Figure 1 | Nemotron 3 models (e.g., Nemotron Nano 3) leverage a hybrid Mamba-Transformer MoE architecture consisting predominantly of interleaved Mamba-2 and MoE layers, with a select few self attention layers.

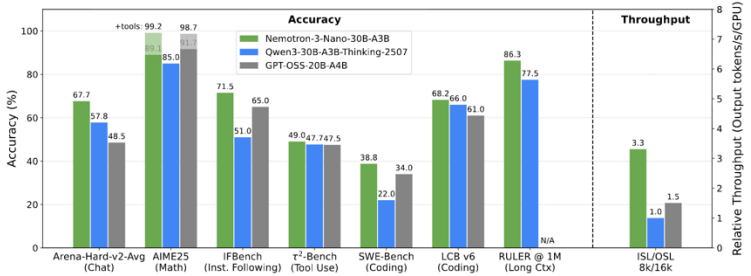


Figure 2 | The hybrid Mamba-Transformer MoE architecture used by Nemotron 3 models can achieve state-of-the-art accuracy on leading reasoning benchmarks and ultra-long-context tasks while providing throughput improvements over similarly sized Transformer MoEs. For details, please see the Nemotron Nano 3 technical report.

Mamba attention hybrid (3-1 ish) – comparable (or better) pref to other similar models

Gated delta net (and friends)

Let's generalize things further – gate the input and selectively erase the state.

vtTD是一个残差连接项。vtT是当前位置的value，D是一个gate

$$S_t = \gamma_t S_{t-1} + k_t v_t^\top \quad \text{and} \quad y_t = q_t^\top S_t + v_t^\top D \quad \text{with } \gamma_t = f(x_t) \text{(Mamba-2)}$$

$$S_t = \gamma_t (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top \quad \text{and}$$
$$y_t = q_t^\top S_t \text{ with } \gamma_t = f(x_t), \beta_t = f(x_t) \text{ (Gated Delta Net)}$$

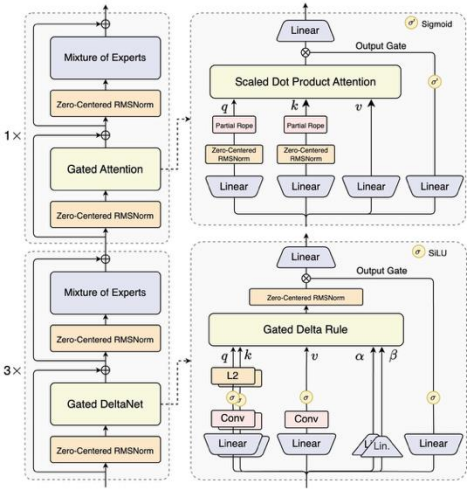
The gated delta net adds a 'no input operation' gate ($\beta = 0$)

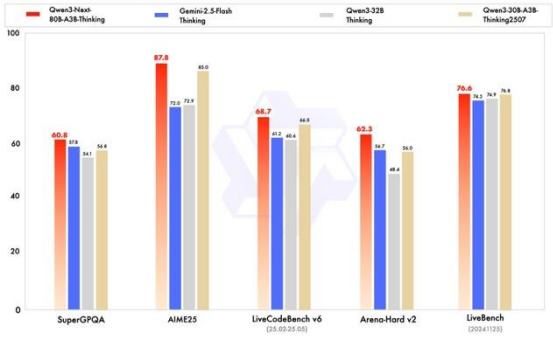
And *erases* anything in the direction of the current key ($I - \beta_t k_t k_t^\top$)

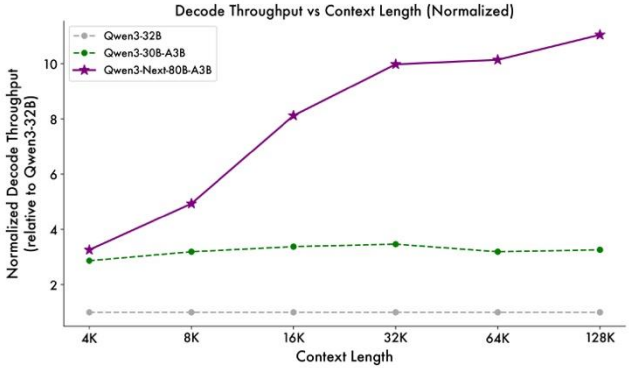
Close relationships to various fast weight programming / test time training ideas.

#TODO：这里说的erase，还有no input operation等我不理解，直接详细解释下gated delta net的思想，和相比mamba-2的改进

Qwen 3.5 / Qwen Next







The newest qwen are 3-1 GDN / Attention hybrids.

Once again, pretty reasonable performance with good inference characteristics

Hybrid performance

ByteDance | Seed UC SANTA CRUZ

A Systematic Analysis of Hybrid Linear Attention

Dustin Wang*[1], Rui-Jie Zhu*[1,2],
Steven Abreu[3], Yong Shan[2], Taylor Kergan[1], Yuqi Pan[2,4], Yuhong Chou[5], Zheng Li[2],
Ge Zhang[2,6,†], Wenhao Huang[2], Jason Eshraghian[1,†]

[1]UC Santa Cruz, [2]ByteDance Seed, [3]University of Groningen, [4]CASIA, [5]PolyU, [6]M-A-P

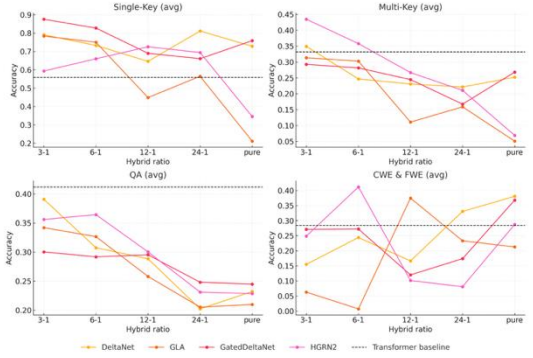
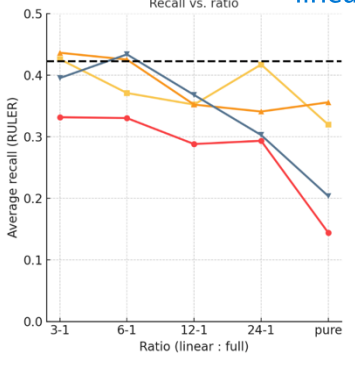


Figure 4 RULER sub task results based on ratio. RetNet and HGRN model families are omitted as their recall benchmark results were insignificant.

Model	Update rule ($\mathbf{S}_t$)	Read-out (o_t)
Vector-valued hidden state (classical / gated RNNs)		
HGRN[4]	$\boldsymbol{h}_t = \boldsymbol{\alpha}_t \odot \boldsymbol{h}_{t-1} + (1-\boldsymbol{\alpha}_t) \odot \boldsymbol{v}_t$	$\boldsymbol{o}_t = \boldsymbol{h}_t \odot \boldsymbol{q}_t$
Hawk (RG-LRU)[33]	$\boldsymbol{h}_t = \boldsymbol{r}_t \, \boldsymbol{h}_{t-1} + \boldsymbol{i}_t \odot \boldsymbol{x}_t$	$\boldsymbol{o}_t = \boldsymbol{h}_t \odot \boldsymbol{q}_t$
Matrix-valued state via outer products		
RetNet/Lightning [11, 17]	$\mathbf{S}_t = \gamma \mathbf{S}_{t-1} + \boldsymbol{v}_t \boldsymbol{k}_t^\top$	$\boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t$
GLA[34]	$\mathbf{S}_t = \mathbf{S}_{t-1} \odot (\mathbf{1}\boldsymbol{\alpha}_t^\top) + \boldsymbol{v}_t \boldsymbol{k}_t^\top$	$\boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t$
Mamba-2[35]	$\mathbf{S}_t = \gamma_t \, \mathbf{S}_{t-1} + \boldsymbol{v}_t \boldsymbol{k}_t^\top$	$\boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t$
RWKV-6[36]	$\mathbf{S}_t = \mathbf{S}_{t-1} \mathrm{Diag}(\boldsymbol{\alpha}_t) + \boldsymbol{v}_t \boldsymbol{k}_t^\top$	$\boldsymbol{o}_t = (\mathbf{S}_{t-1} + (\boldsymbol{d} \odot \boldsymbol{v}_t)\boldsymbol{k}_t^\top)\boldsymbol{q}_t$
HGRN-2/MetaLA[4, 37]	$\mathbf{S}_t = \mathbf{S}_{t-1} \mathrm{Diag}(\boldsymbol{\alpha}_t) + \boldsymbol{v}_t (1-\boldsymbol{\alpha}_t)^\top$	$\boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t$
Delta-rule / controlled-forgetting family		
DeltaNet[5]	$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top) + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^\top$	$\boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t$
Gated DeltaNet[19]	$\mathbf{S}_t = \alpha_t \, \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top) + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^\top$	$\boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t$

横轴从左往右是增加
linear attn（non-full attn）占比



很多non-full注意力只加入几层的时候，表现还挺好的，但是一旦多加，表现下降很明显

Not many controlled ablations, but some evidence of low losses at small hybrid ratios

Alternative to hybrids: sparse adaptation

Instead of attending to every token, do sparse attention (DSA) deepseek sparse attention

#TODO：DSA的详细解释和数学原理

Prototype of DSA. The prototype of DSA primarily consists of two components: a lightning indexer and a fine-grained token selection mechanism.

The **lightning indexer** computes the index score $I_{t,s}$ between the query token $\mathbf{h}_t \in \mathbb{R}^d$ and a preceding token $\mathbf{h}_s \in \mathbb{R}^d$, determining which tokens to be selected by the query token:

$$I_{t,s} = \sum_{j=1}^{H^I} w_{t,j}^I \cdot \mathrm{ReLU}\left(\mathbf{q}_{t,j}^I \cdot \mathbf{k}_s^I\right), \tag{1}$$

where H^I denotes the number of indexer heads; $\mathbf{q}_{t,j}^I \in \mathbb{R}^{d^I}$ and $w_{t,j}^I \in \mathbb{R}$ are derived from the query token $\mathbf{h}_t$; and $\mathbf{k}_s^I \in \mathbb{R}^{d^I}$ is derived from the preceding token $\mathbf{h}_s$. We choose ReLU as the activation function for throughput consideration. Given that the lightning indexer has a small number of heads and can be implemented in FP8, its computational efficiency is remarkable.

Given the index scores $\{I_{t,s}\}$ for each query token $\mathbf{h}_t$, our **fine-grained token selection mechanism** retrieves only the key-value entries $\{\mathbf{c}_s\}$ corresponding to the top-k index scores. Then, the attention output $\mathbf{u}_t$ is computed by applying the attention mechanism between the query token $\mathbf{h}_t$ and the sparsely selected key-value entries $\{\mathbf{c}_s\}$:

$$\mathbf{u}_t = \mathrm{Attn}\left(\mathbf{h}_t, \left\{\mathbf{c}_s \,\middle|\, I_{t,s} \in \text{Top-k}(I_{t,:})\right\}\right). \tag{2}$$

训练过程中：可以先pretrain一个不带这个lts的正常的transformer，然后在另外一个extention stage，用预训练好的LLM加上这个lts做一轮新的训练。

simple idea：
先从整个context中选一个subset，然后再在subset上做full-attn。

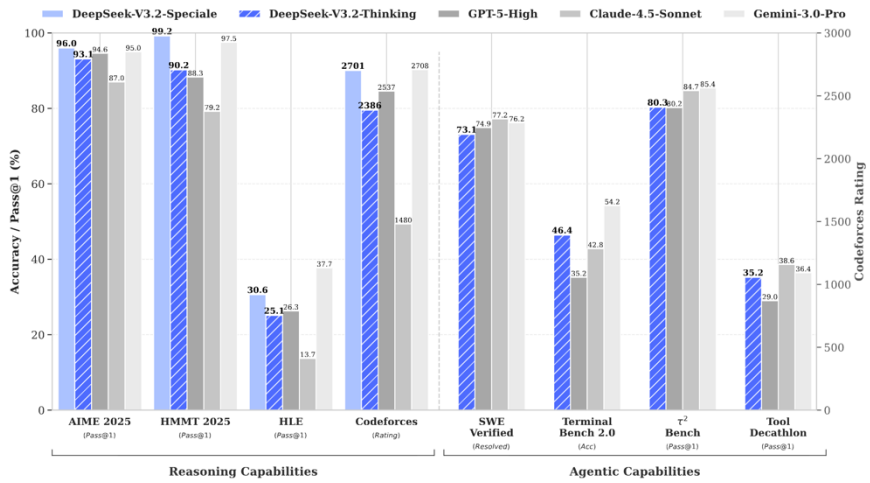
top-k selection是核心思想

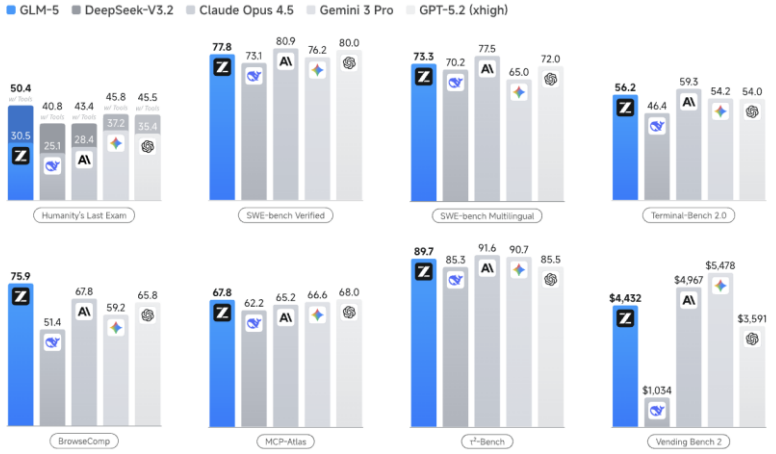
#TODO：这里的lts的生成，也是二次方复杂度的对吗？那么是如何节省时间的？？首先是可以做很多system z-trick优化；其次已经选了top-k，所以是k^2复杂度而不是n^2，这也是常数的重要性（这里不理解）

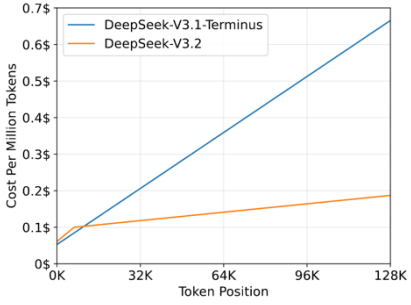
The indexer can be very lightweight, giving significant gains

Can be ‘post hoc’ adapted after dense short context pretraining

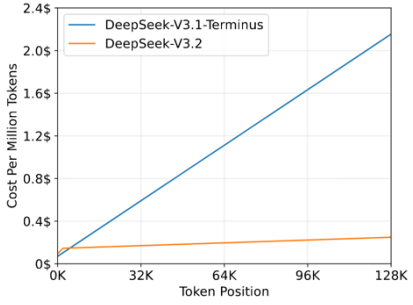
DSA – Deepseek Sparse Attention (v3.2, GLM5)







(a) Prefilling



(b) Decoding

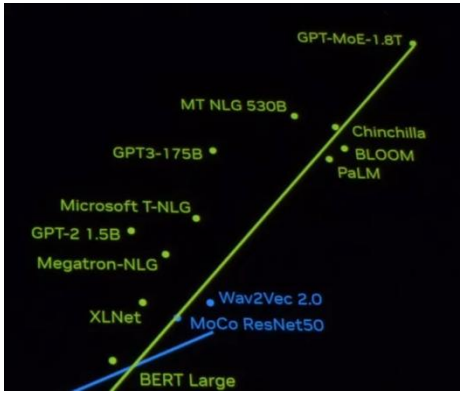
full-DSA training表现最好

Table 6: RULER benchmark results for the GLM-4.7-Flash with DSA. The warmup-only variant trains only the indexer while keeping the base model frozen, the full DSA variant jointly trains both for 150B tokens.

	4K	8K	16K	32K	64K	128K
GLM-4.7-Flash	97.44	96.72	95.83	92.96	85.34	79.21
GLM-4.7-Flash + DSA warmup	97.51	96.54	95.40	90.09	84.05	71.35
GLM-4.7-Flash + DSA	96.77	96.25	96.69	93.45	87.06	78.86

DeepSeek-V3.2在表现和其他LLM差不多的情况下，成本更低

Mixture of experts



GPT4 (?)

Mistral AI @MistralAI

magnet:?xt=urn:btih:9238b09245d0d8cd915be09927769d5f7584c1c9&dn=mixtral-8x22b&tr=udp%3A%2F%2Fopen.demonii.com%3A1337%2Fannounce&tr=http%3A%2F%2Ftracker.opentrackr.org%3A1337%2Fannounce

Grok

deepseek

DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models

deepseek

DeepSeek-V3 Technical Report



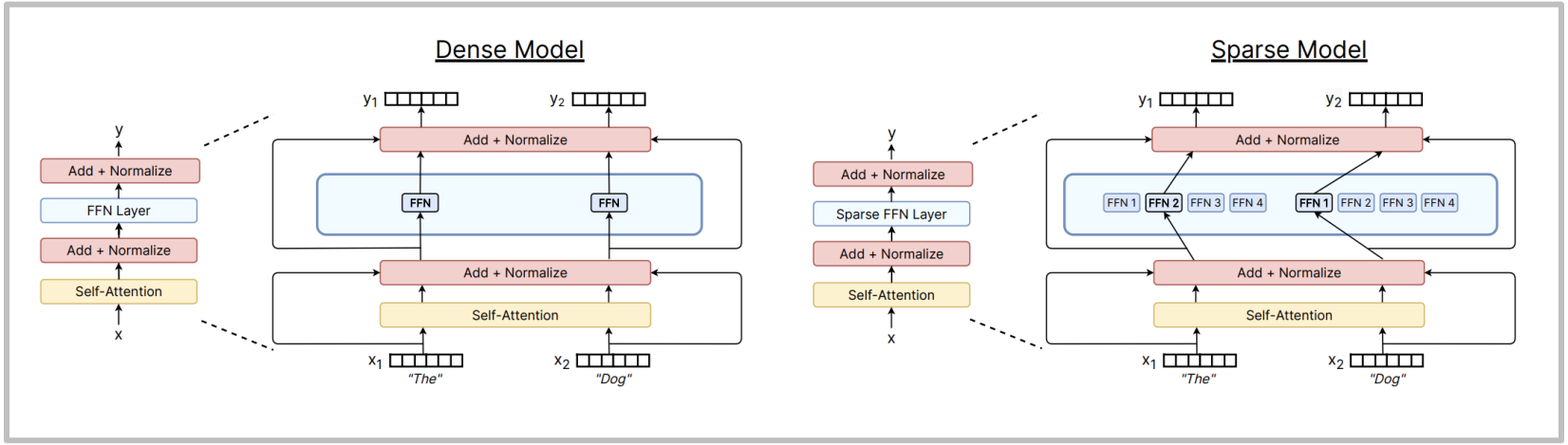
OLMoE: Open Mixture-of-Experts Language Models

Niklas Muennighoff[ca] Luca Soldaini[a] Dirk Groeneveld[a] Kyle Lo[a] Jacob Morrison[a]
Sewon Min[a] Weijia Shi[w] Pete Walsh[a] Oyvind Tafjord[a] Nathan Lambert[a]
Yuling Gu[a] Shane Arora[a] Akshita Bhagia[a] Dustin Schwenk[a] David Wadden[a]
Alexander Wettig[ap] Binyuan Hui Tim Dettmers[a] Douwe Kiela[c] Ali Farhadi[aw]
Noah A. Smith[aw] Pang Wei Koh[aw] Amanpreet Singh[c] Hannaneh Hajishirzi[aw]
[a]Allen Institute for AI [c]Contextual AI [w]University of Washington [p]Princeton University
n.muennighoff@gmail.com hannah@allenai.org

What's a MoE?

per-token FFN

4倍参数量，但是前向和反向传播的flops
和dense model相同。#TODO 为啥相同?



[Fedus et al 2022]

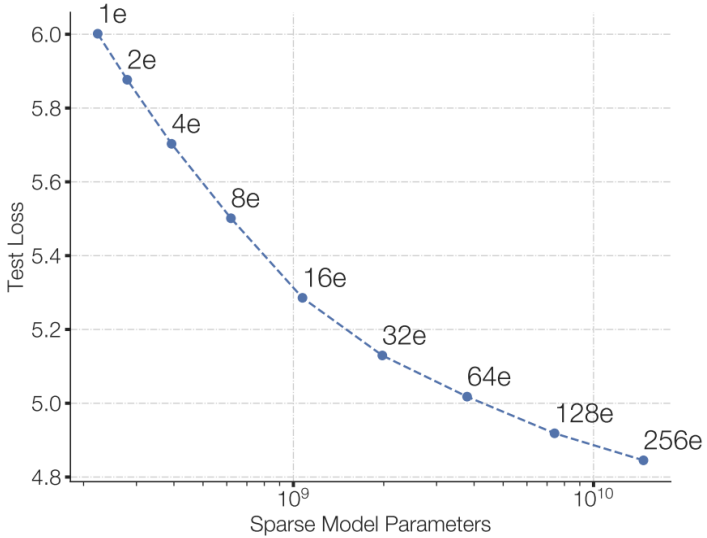
Replace big feedforward with (many) big feedforward networks and a selector layer

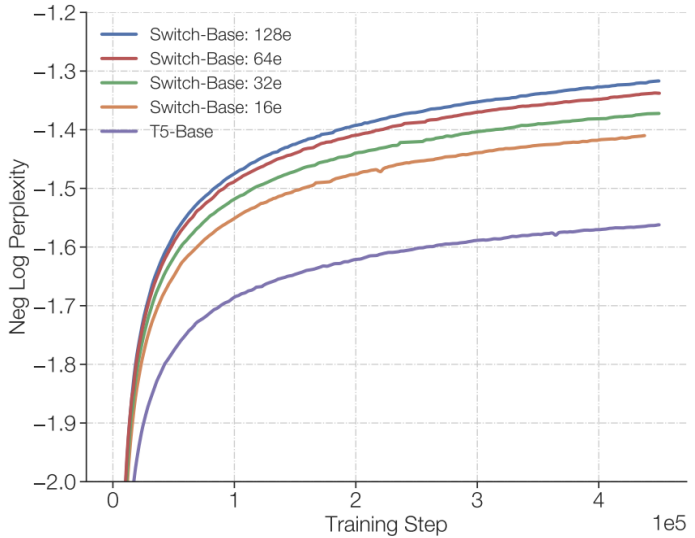
You can increase the # experts without affecting FLOPs

Why are MoEs getting popular?

Same FLOP, more param does better

MoE allows us to increase parameters without increasing flops



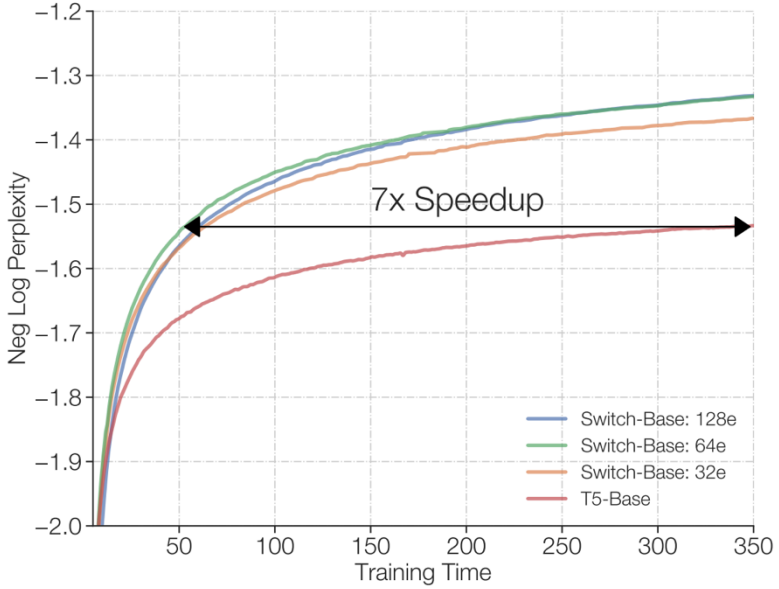


这张图中的每个点的flops都是相同的，一个可能：总专家数量增大，但是激活的专家数量不变

[Fedus et al 2022]

Why are MoEs getting popular?

Faster to train MoEs



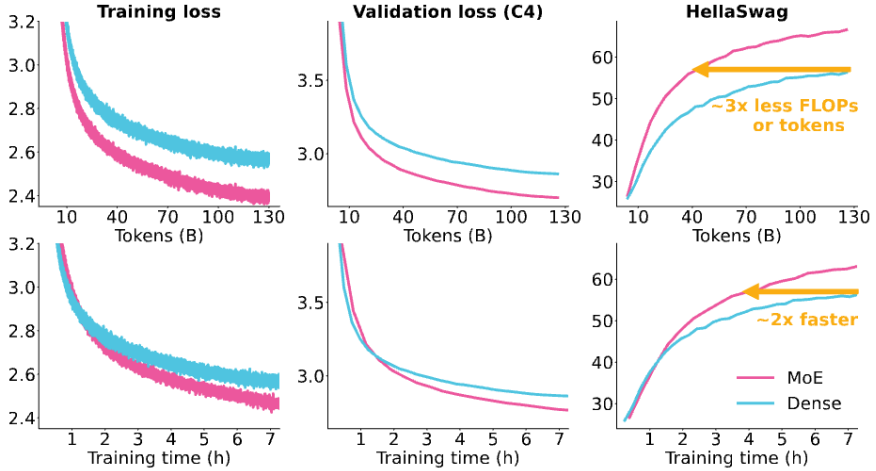
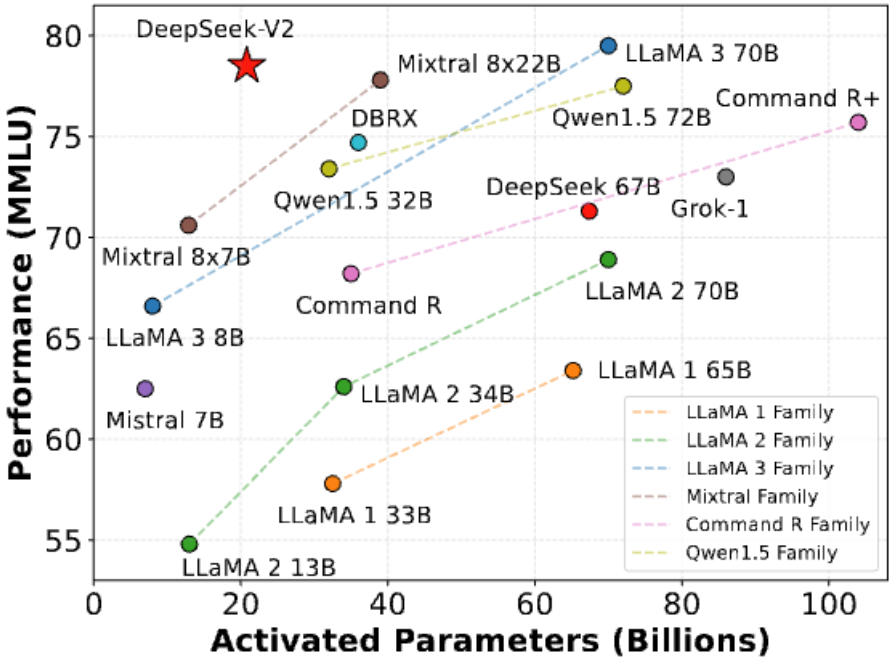


Figure 4: **MoE vs. Dense.** We train a 1.3B parameter dense model and a 1.3B active, 6.9B total parameter MoE model, each on 128 H100 GPUs. Apart from MoE-related changes, we train both

[OlMoE]

Why are MoEs getting popular?

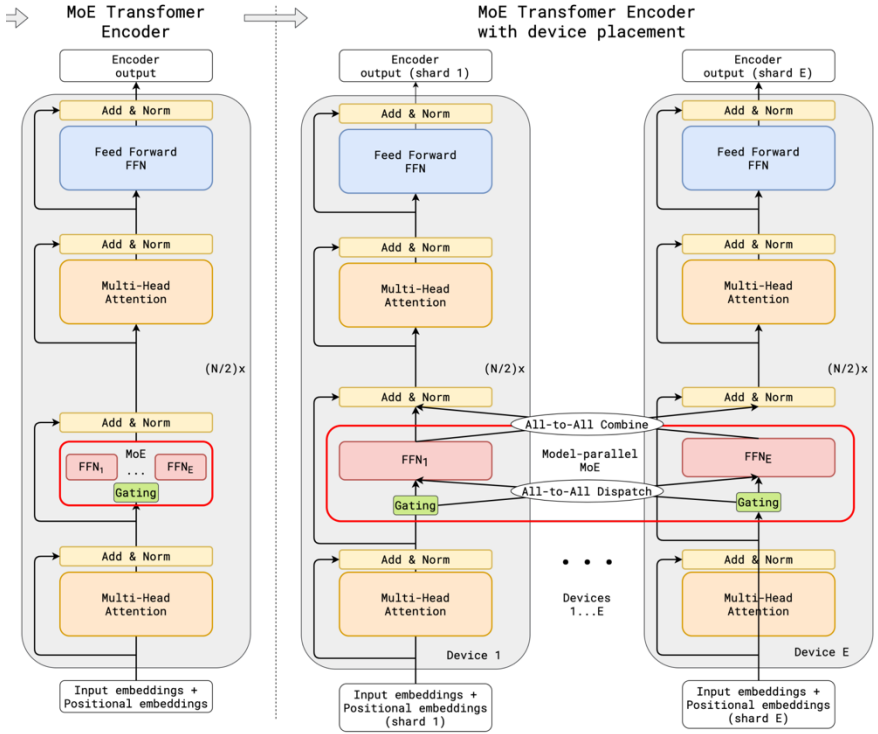


Highly competitive vs dense equivalents

Why are MoEs getting popular?

Parallelizable to many devices

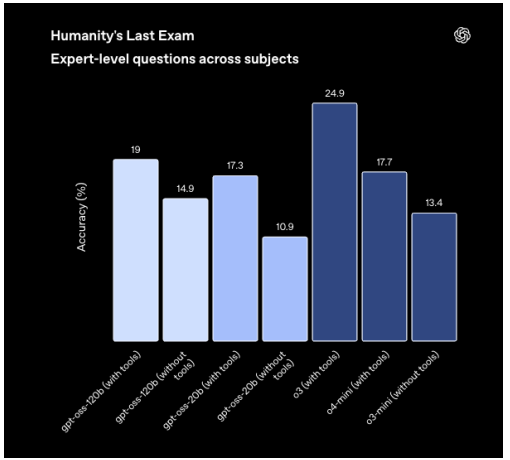
expert parallel



#TODO：详细解释下expert parallel？到底是将什么东西并行了？只并行专家，还是将整个模型都并行了？为什么需要并行？训练还是推理的时候并行？具体原理是什么？

Some MoE results – from the west

Category Benchmark	Llama 4 Maverick	Gemini 2.0 Flash	DeepSeek v3.1	GPT-4o
Inference Cost Cost per 1M input & output tokens (3:1 blended)	$0.19-$0.49[5]	**$0.17**	$0.48	$4.38
Image Reasoning MMMU	**73.4**	71.7	No multimodal support	69.1
MathVista	**73.7**	73.1		63.8
Image Understanding ChartQA	**90.0**	88.3		85.7
DocVQA (test)	**94.4**	—		92.8
Coding LiveCodeBench (10/01/2024-02/01/2025)	43.4	34.5	**45.8/49.2**[3]	32.3[3]
Reasoning & Knowledge MMLU Pro	80.5	77.6	**81.2**	—
GPQA Diamond	**69.8**	60.1	68.4	53.6



MoEs are most of the highest-performance open models, and are quite quick

Earlier MoE results from Chinese groups – Qwen

Chinese LLM companies are also doing quite a bit of MoE work on the smaller end

Model	MMLU	GSM8K	HumanEval	Multilingual	MT-Bench
Mistral-7B	64.1	47.5	27.4	40.0	7.60
Gemma-7B	64.6	50.9	32.3	-	-
Qwen1.5-7B	61.0	62.5	36.0	45.2	7.60
DeepSeekMoE 16B	45.0	18.8	26.8	-	6.93
Qwen1.5-MoE-A2.7B	62.5	61.5	34.2	40.8	7.17

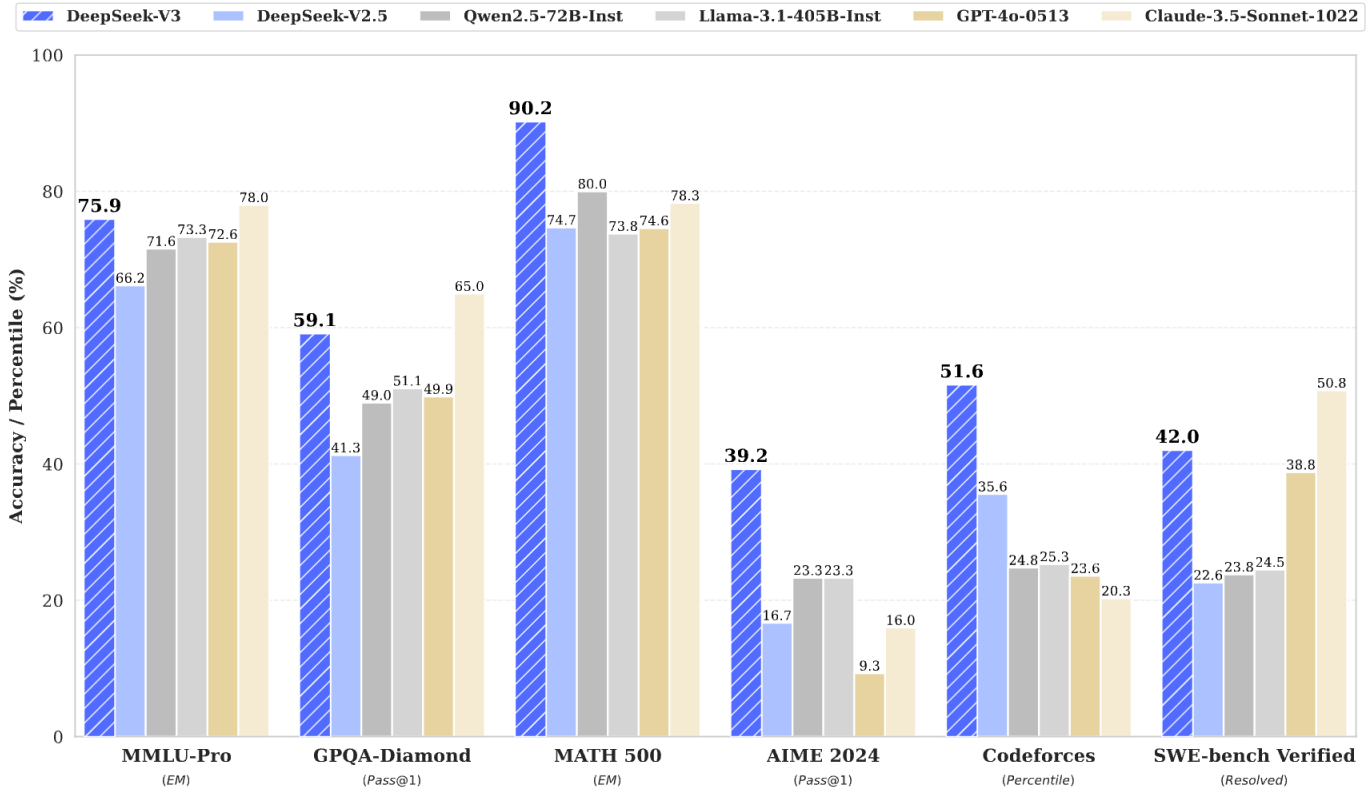
Model	#Parameters	#(Activated) Parameters
Mistral-7B	7.2	7.2
Qwen1.5-7B	7.7	7.7
Gemma-7B	8.5	7.8
DeepSeekMoE 16B	16.4	2.8
Qwen1.5-MoE-A2.7B	14.3	2.7

Earlier MoE results from Chinese groups - DeepSeek

There's also some good recent ablation work on MoEs showing they're generally good

Metric	# Shot	Dense	Hash Layer	Switch
# Total Params	N/A	0.2B	2.0B	2.0B
# Activated Params	N/A	0.2B	0.2B	0.2B
FLOPs per 2K Tokens	N/A	2.9T	2.9T	2.9T
# Training Tokens	N/A	100B	100B	100B
Pile (Loss)	N/A	2.060	1.932	1.881
HellaSwag (Acc.)	0-shot	38.8	46.2	49.1
PIQA (Acc.)	0-shot	66.8	68.4	70.5
ARC-easy (Acc.)	0-shot	41.0	45.3	45.9
ARC-challenge (Acc.)	0-shot	26.0	28.2	30.2
RACE-middle (Acc.)	5-shot	38.8	38.8	43.6
RACE-high (Acc.)	5-shot	29.0	30.0	30.9
HumanEval (Pass@1)	0-shot	0.0	1.2	2.4
MBPP (Pass@1)	3-shot	0.2	0.6	0.4
TriviaQA (EM)	5-shot	4.9	6.5	8.9
NaturalQuestions (EM)	5-shot	1.4	1.4	2.5

Recent MoE results – DeepSeek v3



Why haven’t MoEs been more popular?

Infrastructure is complex / advantages on multi node

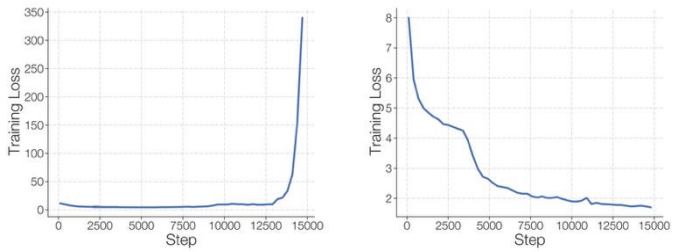
#TODO：这段话没看懂

> At a high level, sparsity is good when you have many accelerators (e.g. GPU/TPU) to host all the additional parameters that comes when using sparsity. Typically models are trained using data-parallelism where different machines will get different slices of the training/inference data. The machines used for operating on the different slices of data can now be used to host many more model parameters. Therefore, sparse models are good when training with data parallelism and/or have high throughput while serving: training/serving on many machines which can host all of the parameters.

[Fedus et al 2022]

Training objectives are somewhat heuristic (and sometimes unstable)

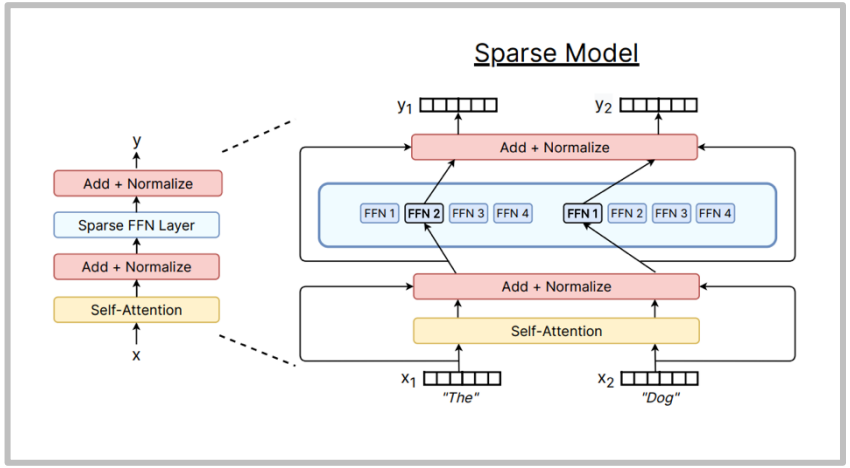
Sparse models often suffer from training instabilities (Figure 1) worse than those observed in standard densely-activated Transformers.



[Zoph et al 2022]

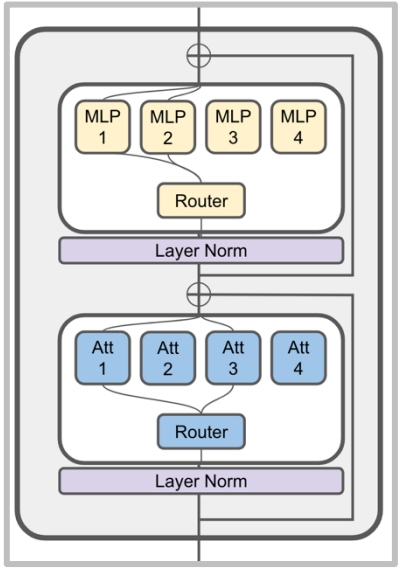
What MoEs generally look like

Typical: replace MLP with MoE layer



Less common: MoE for attention heads

这个应该是让router选择attention head



[ModuleFormer, JetMoE]

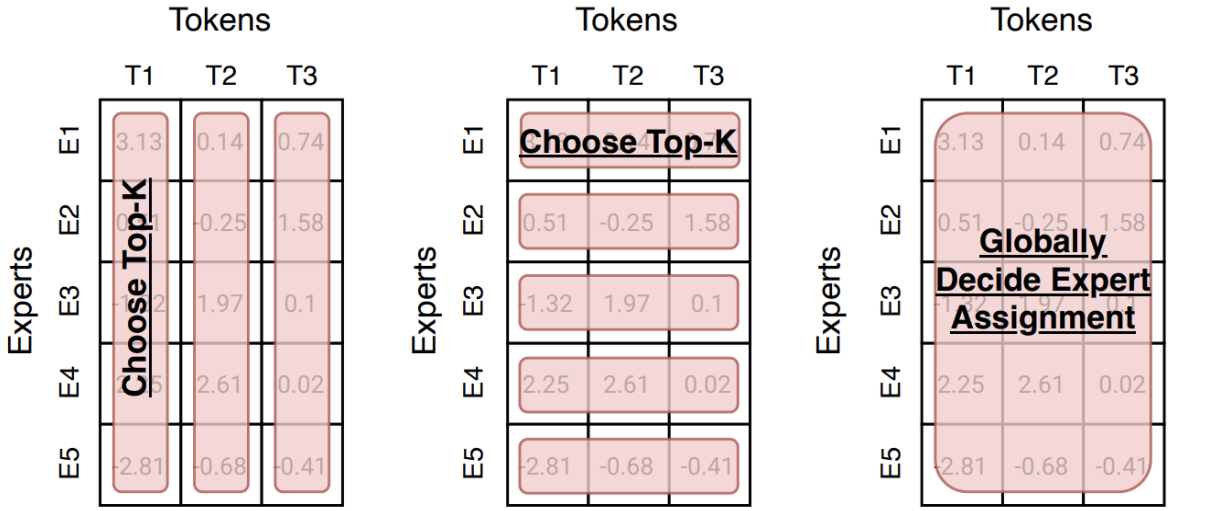
MoE – what varies?

- Routing function
- Expert sizes — given budget, fewer experts with more params per expert, or vice versa
- Training objectives

Routing function - overview

#TODO：详细解释下这三种，特别是expert pick token，这个具体是啥意思？输入输出形状？通过一个具体数值例子，对比这三者的表现。

Many of the routing algorithms boil down to ‘choose top k’



Token chooses expert

这个是大多数MoE做的

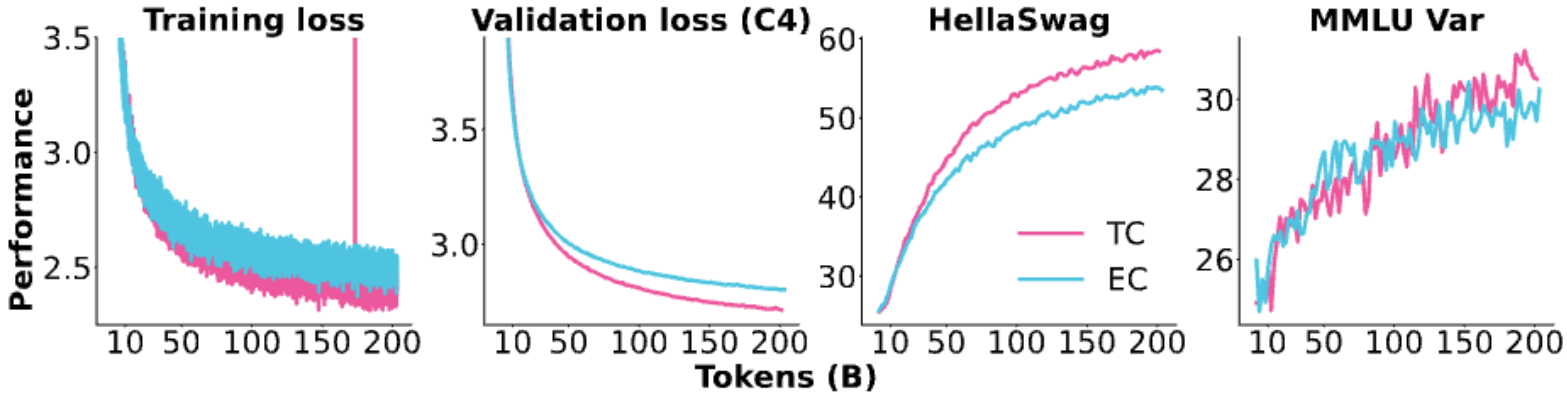
Expert chooses token

Global routing via optimization

[Fedus et al 2022]

Routing type

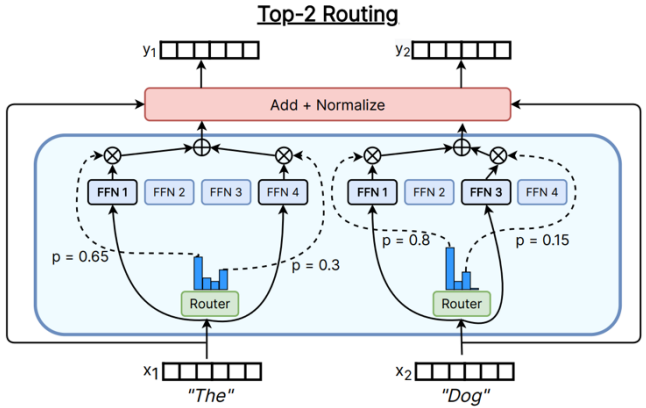
Almost all the MoEs do a standard ‘token choice topk’ routing. Some recent ablations



红色：token chooses expert（表现更好）
蓝色：expert chooses token

Common routing variants in detail

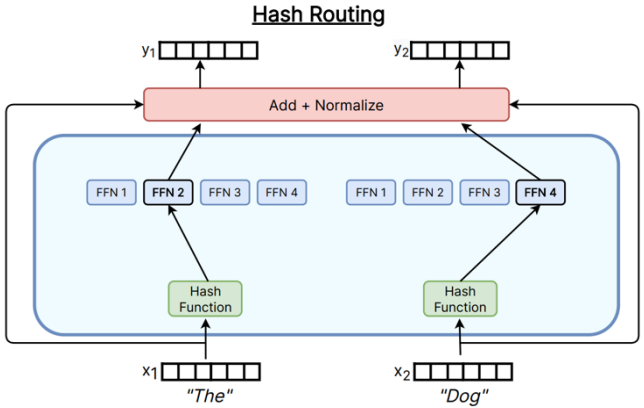
Top-k



Used in *most* MoEs

Switch Transformer (k=1)
Gshard (k=2), Grok (2), Mixtral (2),
Qwen (4), DBRX (4),
DeepSeek (7)

Hashing



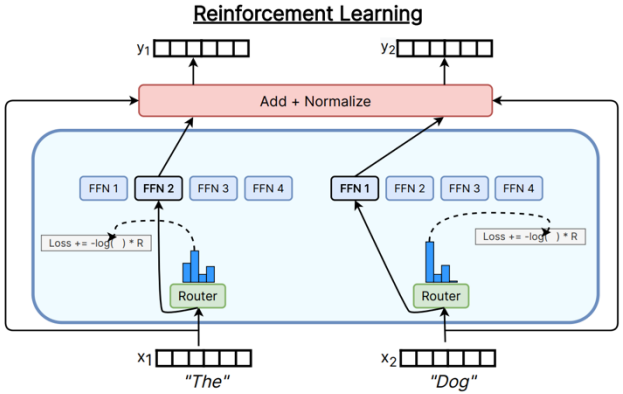
#TODO：hashing具体指的是啥？如何做的？

Common baseline

[Fedus et al 2022]

Other routing methods

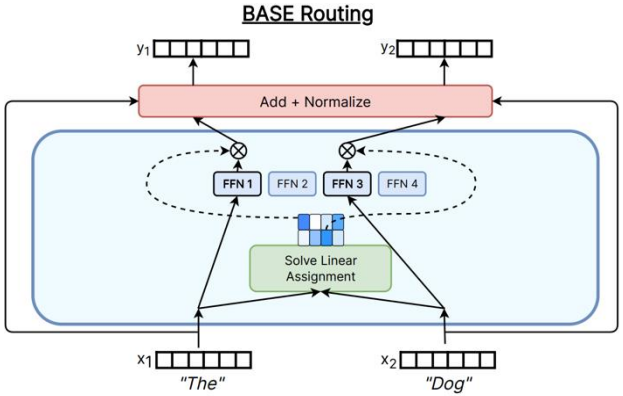
RL to learn routes



Used in some of the earliest work
Bengio 2013, not common now

做RL太过于复杂，而且效果也不比直接做top-k好

Solve a matching problem



#TODO：解释下这个linear assignment具体是啥。

Linear assignment for routing
Used in various papers like Clark ‘22

[Fedus et al 2022]

Top-K routing in detail.

Most papers do the old and classic top-k routing. How does this work?

Gating

$$\mathbf{h}_t^l = \sum_{i=1}^{N} \left(g_{i,t} \, \mathrm{FFN}_i \left(\mathbf{u}_t^l \right) \right) + \mathbf{u}_t^l,$$

This is the DeepSeek (V1-2) router (Grok, Qwen do this too)

$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \mathrm{Topk}(\{s_{j,t} | 1 \leqslant j \leqslant N\}, K), \\ 0, & \text{otherwise}, \end{cases}$$

$$s_{i,t} = \mathrm{Softmax}_i \left(\mathbf{u}_t^{l\,T} \mathbf{e}_i^l \right),$$

Mixtral, DBRX, DeepSeek v3 softmaxes after the TopK

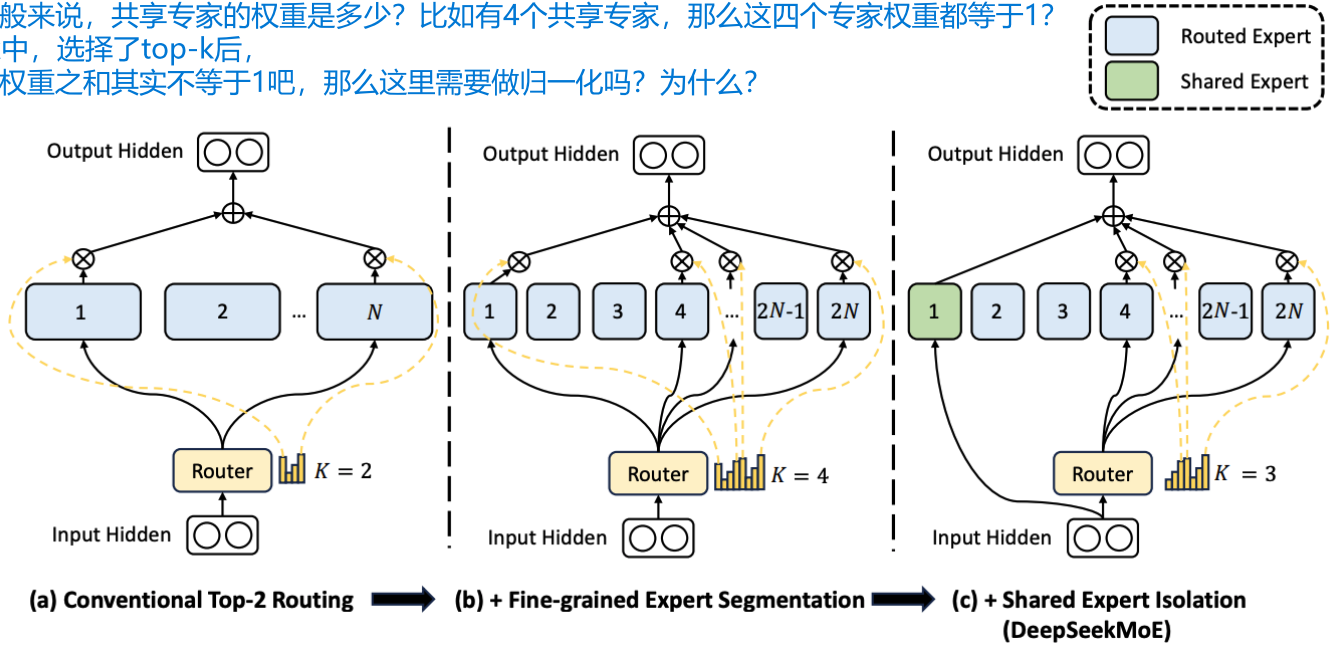
#TODO：详细解释下这里的公式，包括ut，ei等等含义。以及这里的sit是如何得到的。一般来说，MoE的门控网络的权重不是应该是输入过一个MLP+softmax得到的吗？这里为啥是ut和ei做inner product？这是什么逻辑？

Gates selected by a logistic regressor

[Dai et al 2024]

Recent variations from DeepSeek and other Chinese LMs

#TODO：一般来说，共享专家的权重是多少？比如有4个共享专家，那么这四个专家权重都等于1？
另外，top-k中，选择了top-k后，
这k个专家的权重之和其实不等于1吧，那么这里需要做归一化吗？为什么？



Smaller, larger number of experts + a few shared experts that are always on.

intuition：如果没有共享专家，那么每个专家都会有一部分做"common modelling"，
加入共享专家让各个任务的专家specialize even more。

(Used in DeepSeek / Qwen, originally from DeepSpeed MoE)

Various ablations from the DeepSeek paper

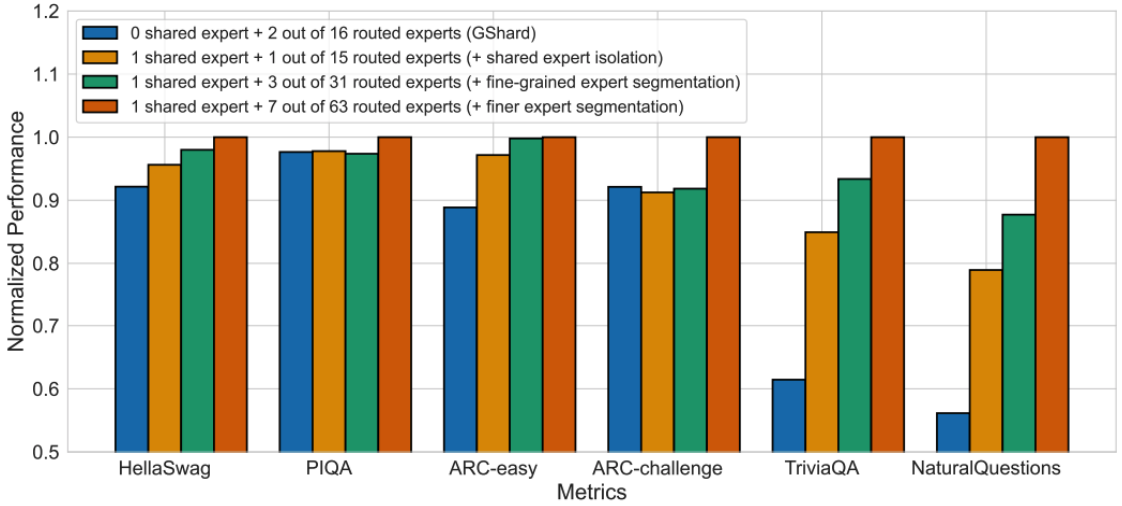
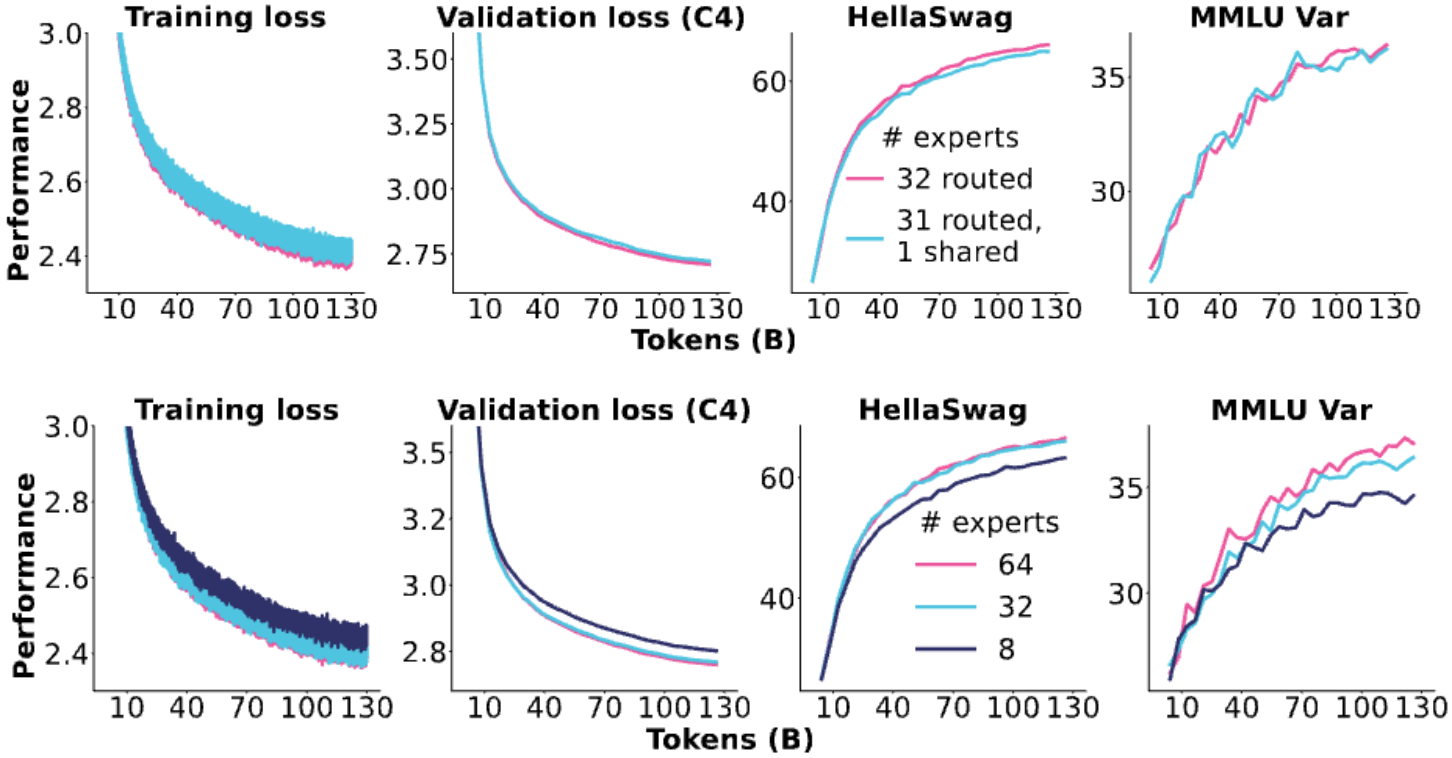


Figure 3 | Ablation studies for DeepSeekMoE. The performance is normalized by the best performance for clarity in presentation. All compared models have the same number of parameters and activated parameters. We can find that fine-grained expert segmentation and shared expert isolation both contribute to stronger overall performance.

More experts, shared experts all seem to generally help

Ablations from OlMoE

Gains from fine-grained experts, none from shared experts.



Expert routing setups for recent MoEs

#TODO：这里的fine-grained ratio指的是啥?

Model	Routed	Active	Shared	Fine-grained ratio
GShard	2048	2	0	
Switch Transformer	64	1	0	
ST-MOE	64	2	0	
Mixtral	8	2	0	
DBRX	16	4	0	
Grok	8	2	0	
DeepSeek v1	64	6	2	1/4
Qwen 1.5	60	4	4	1/8
DeepSeek v3	256	8	1	1/14
OlMoE	64	8	0	1/8
MiniMax	32	2	0	~1/4
Llama 4 (maverick)	128	1	1	1/2

比如DeepSeek v3，则共享1个，剩下的255个里面激活8个

How do we train MoEs?

Major challenge: we need sparsity for training-time efficiency…

But sparse gating decisions are not differentiable!

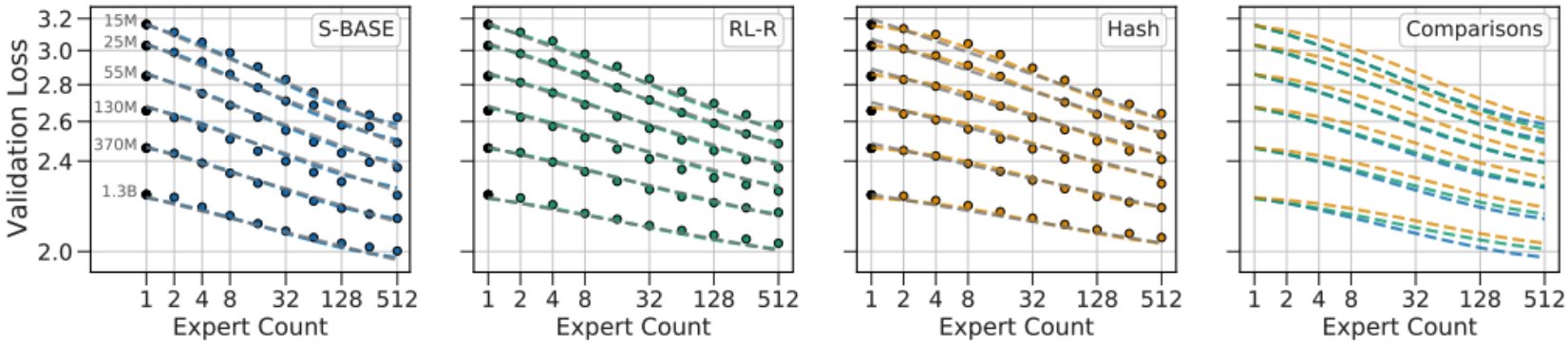
Solutions?

1. Reinforcment learning to optimize gating policies
2. Stochastic perturbations 随机扰动
3. Heuristic ‘balancing’ losses. 启发式的”平衡损失“

Guess which one people use in practice?

RL for MoEs

RL via REINFORCE does work, but not so much better that it's a clear win



(REINFORCE baseline approach, Clark et al 2020)

RL is the 'right solution' but gradient variances and complexity means it's not widely used

Stochastic approximations

也是一种将不可微的topk选择变成可微的方法

#TODO：这个式子和前面几页ppt的公式是否存在不同？前面的是门控网络的输出先softmax，再选topk；这里是先选topk，再softmax?

$$G(x) = Softmax(KeepTopK(H(x), k))$$

$$H(x)_i = (x \cdot W_g)_i + StandardNormal() \cdot Softplus((x \cdot W_{noise})_i)$$

$$KeepTopK(v, k)_i = \begin{cases} v_i & \text{if } v_i \text{ is in the top } k \text{ elements of } v. \\ -\infty & \text{otherwise.} \end{cases}$$

From Shazeer et al 2017 – routing decisions are *stochastic* with gaussian perturbations.

1. This naturally leads to experts that are a bit more robust.
2. The softmax means that the model learns how to rank K experts

#TODO：详细解释下这里的stochastic方法，目标是什么？这个是如何实现目标的？这里的robust是什么意思？第二点learn how to rank 又是啥意思?

Stochastic approximations

```
if is_training:
    # Add noise for exploration across experts.
    router_logits += mtf.random_uniform(shape=router_logits.shape, minval=1-eps, maxval=1+eps)

# Convert input to softmax operation from bfloat16 to float32 for stability.
router_logits = mtf.to_float32(router_logits)

# Probabilities for each token of what expert it should be sent to.
router_probs = mtf.softmax(router_logits, axis=-1)
```

Stochastic jitter in Fedus et al 2022. This does a uniform multiplicative perturbation for the same goal of getting less brittle experts. This was later removed in Zoph et al 2022

Method	Fraction Stable	Quality (↑)
Baseline	4/6	**-1.755** ±0.02
Input jitter (10^{-2})	3/3	-1.777 ±0.03
Dropout (0.1)	3/3	-1.822 ±0.11

Heuristic balancing losses

#TODO：如果不做balancing，
会导致第一轮的topkexpert在反向传播得到更多的信号，
从而增强这k个dexpert的表现，后续也会趋向选择这k个expert，
这理解对吗？详细解释下。

Another key issue – systems efficiency requires that we use experts evenly..

For each Switch layer, this auxiliary loss is added to the total model loss during training. Given N experts indexed by $i = 1$ to N and a batch $\mathcal{B}$ with T tokens, the auxiliary loss is computed as the scaled dot-product between vectors f and P,

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i \tag{4}$$

where f_i is the fraction of tokens dispatched to expert i,

$$f_i = \frac{1}{T} \sum_{x \in \mathcal{B}} \mathbb{1}\{\text{argmax } p(x) = i\} \tag{5}$$

and P_i is the fraction of the router probability allocated for expert i, [2]

$$P_i = \frac{1}{T} \sum_{x \in \mathcal{B}} p_i(x). \tag{6}$$

From the Switch Transformer [Fedus et al 2022]

放在token chooses expert的范式下理解。
batch中每个样本包含T个token，
针对一个专家，
每个token都有一个打分

The derivative with respect to $p_i(x)$ is $\frac{\alpha N}{T^2} \sum 1_{argmax\ p(x)=i}$,
so more frequent use = stronger downweighting

#TODO：详细解释下这里的公式，以及这里的求导是如何求出来的，导数形式和含义

Example from deepseek (v1-2)

Per-expert balancing – same as the switch transformer

$$\mathcal{L}_{\text{ExpBal}} = \alpha_1 \sum_{i=1}^{N'} f_i P_i, \tag{12}$$

$$f_i = \frac{N'}{K'T} \sum_{t=1}^{T} \mathbb{1}(\text{Token } t \text{ selects Expert } i), \tag{13}$$

$$P_i = \frac{1}{T} \sum_{t=1}^{T} s_{i,t}, \tag{14}$$

#TODO：这里不理解什么叫device balancing。

Per-device balancing – the objective above, but aggregated by device.

$$\mathcal{L}_{\text{DevBal}} = \alpha_2 \sum_{i=1}^{D} f'_i P'_i, \tag{15}$$

$$f'_i = \frac{1}{|\mathcal{E}_i|} \sum_{j \in \mathcal{E}_i} f_j, \tag{16}$$

$$P'_i = \sum_{j \in \mathcal{E}_i} P_j, \tag{17}$$

DeepSeek v3 variation – per-expert biases

Set up a per-expert bias (making it more likely to get tokens) and use online learning

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise.} \end{cases}$$

They call this '**auxiliary loss free balancing**'

#TODO：没看懂这个做法

Complementary Sequence-Wise Auxiliary Loss. Although DeepSeek-V3 mainly relies on the auxiliary-loss-free strategy for load balance, to prevent extreme imbalance within any single sequence, we also employ a complementary sequence-wise balance loss:

$$\mathcal{L}_{\text{Bal}} = \alpha \sum_{i=1}^{N_r} f_i P_i, \tag{17}$$

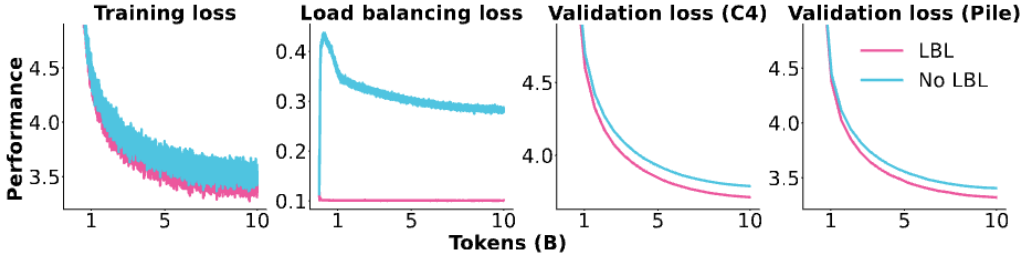
$$f_i = \frac{N_r}{K_r T} \sum_{t=1}^{T} \mathbb{1}\left(s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leqslant j \leqslant N_r\}, K_r)\right), \tag{18}$$

$$s'_{i,t} = \frac{s_{i,t}}{\sum_{j=1}^{N_r} s_{j,t}}, \tag{19}$$

$$P_i = \frac{1}{T} \sum_{t=1}^{T} s'_{i,t}, \tag{20}$$

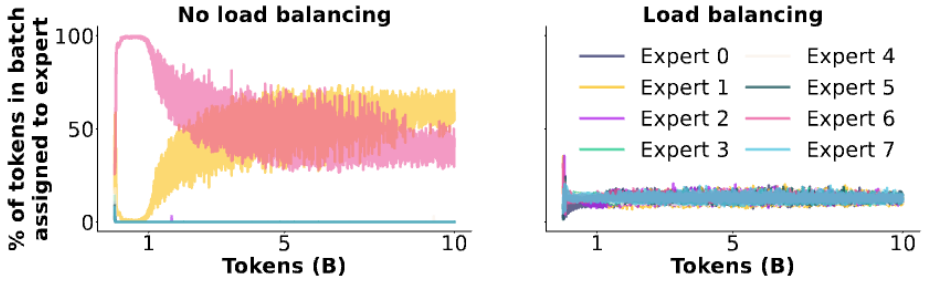
(but the approach is not fully aux loss free..)

What happens when removing load balancing losses?



LB: load-balancing loss

Figure 9: **Impact of applying a load balancing loss (LBL).** The training loss plot excludes the load balancing loss for both models. More results, logs, and configurations: https://wandb.ai/ai2-llm/olmoe/reports/Plot-LBL-vs-No-LBL--Vmlldzo4OTkyNDg4



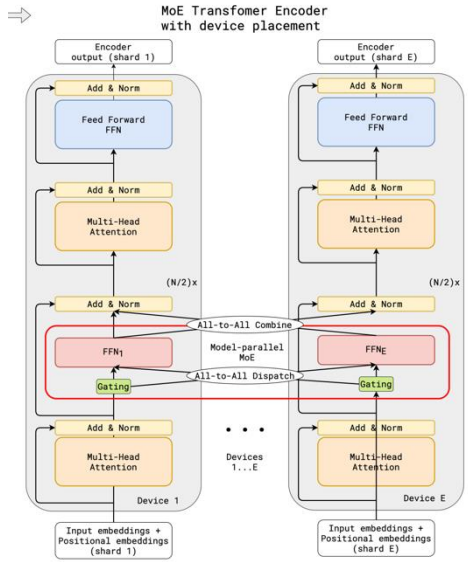
加入LBL后，专家获得的token分配更加平均

Figure 10: **Expert assignment during training when using or not using a load balancing loss for the first MoE layer.** More results, logs, and configurations: https://wandb.ai/ai2-llm/olmoe/reports/Plot-LBL-vs-No-LBL--Vmlldzo4OTkyNDg4

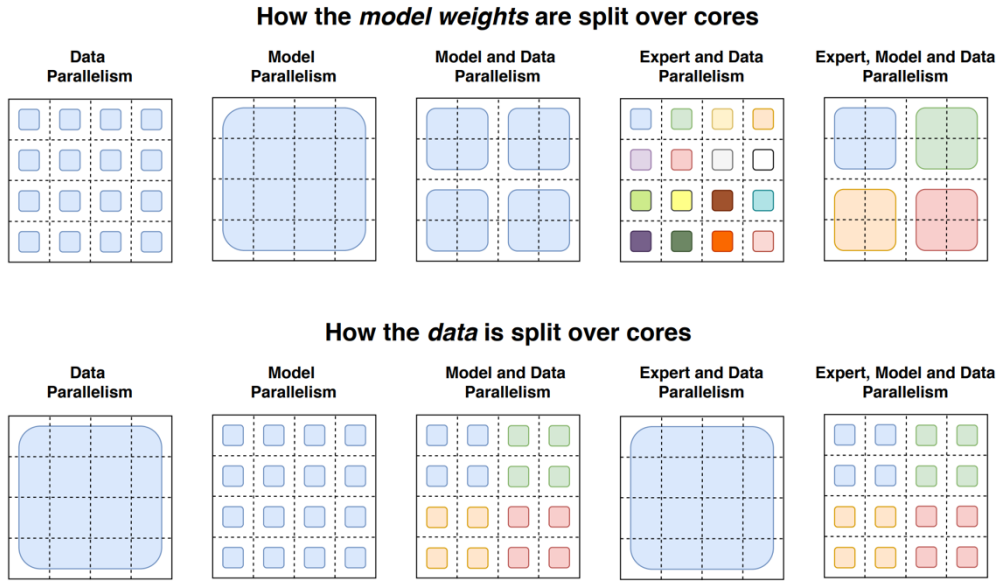
Training MoEs – the systems side

#TODO：详细解释下这里提到的不同的parallelism的方式，以及MoE为啥能很好地并行？MoE是如何并行的？

MoEs parallelize nicely – Each FFN can fit in a device



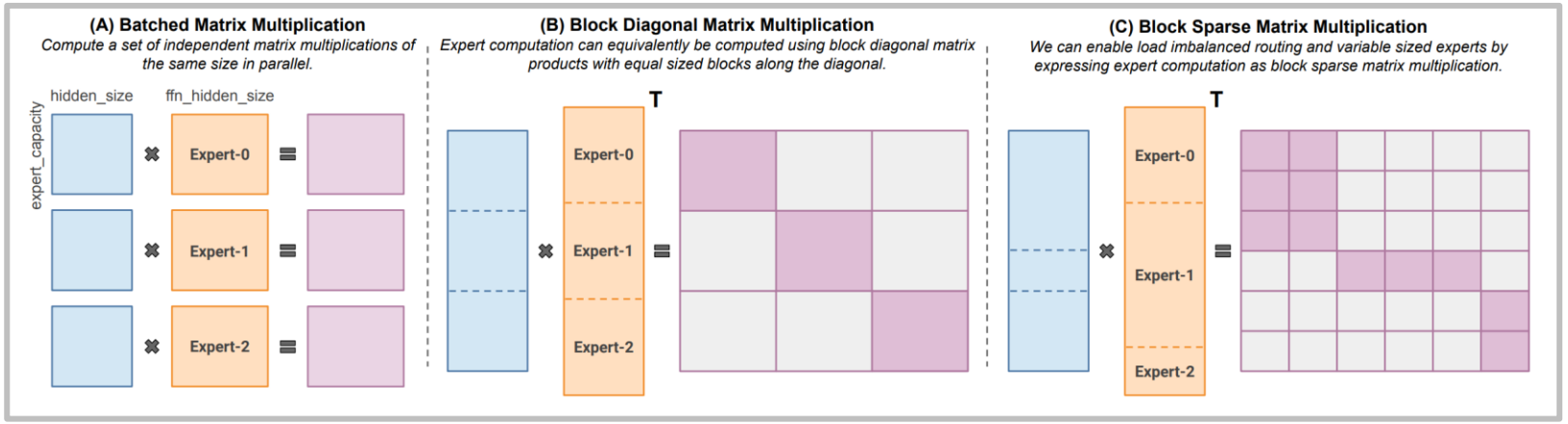
Enables additional kinds of parallelism



Training MoEs – the systems side

MoE routing allows for parallelism, but also some complexities

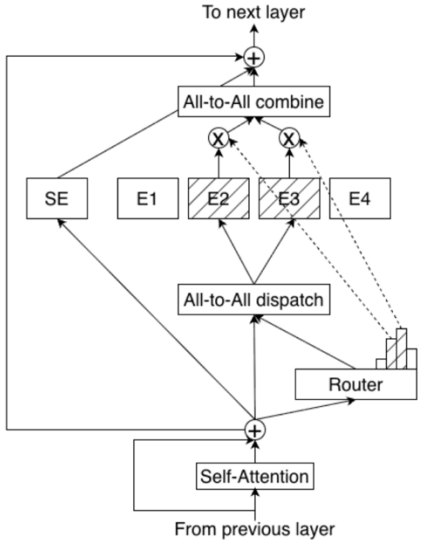
TODO：这里的详细解释？为啥要分块矩阵乘法？没看懂。



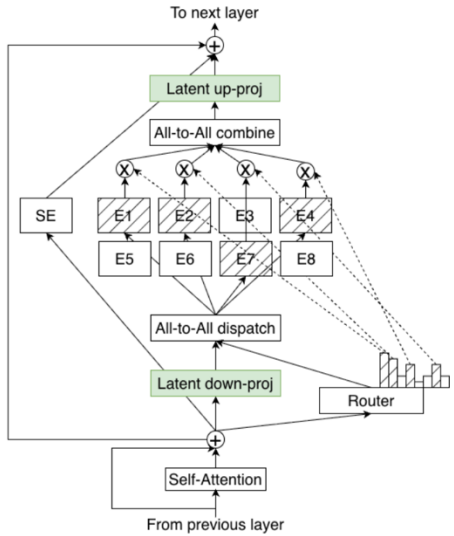
Modern libraries like MegaBlocks (used in many open MoEs) use smarter sparse MMs

MoE parallelism and architecture modifications

2.2. LatentMoE: Hardware-Aware Expert Design for Improved Accuracy per Byte



(a) Standard MoE architecture.



(b) LatentMoE architecture.

#TODO：这页完全没看懂

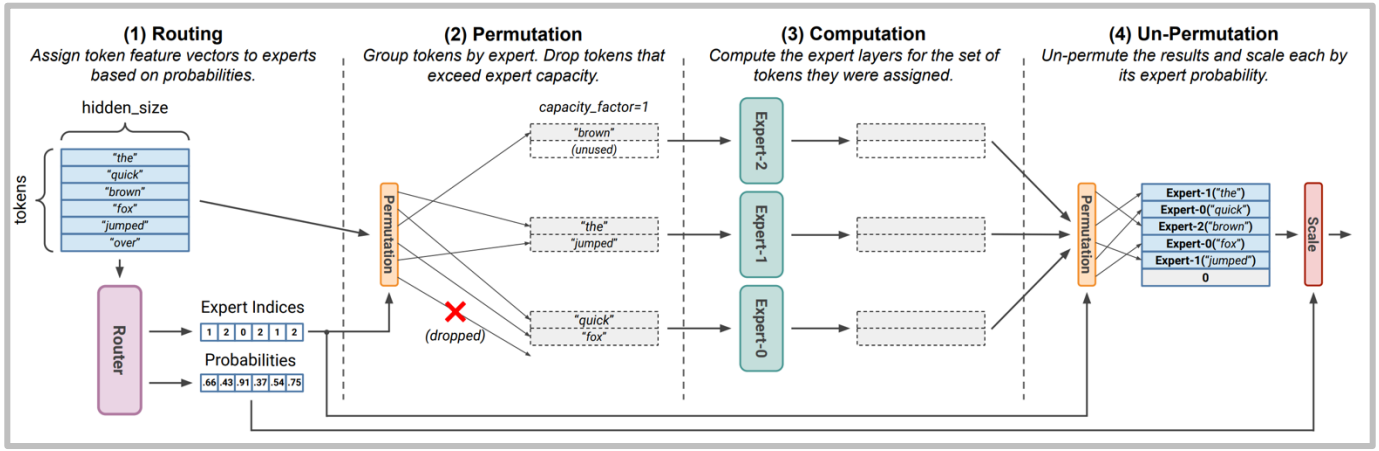
New ideas from Nemotron 3 – down-projecting the activations to reduce communication

Fun side issue – stochasticity of MoE models

MoEs can have additional stochasticity beyond normal models..

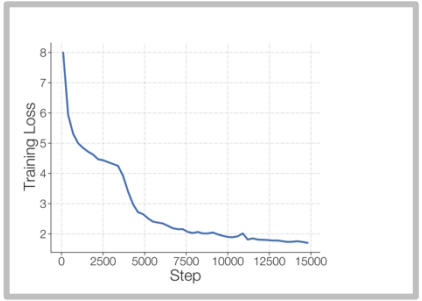
#TODO：expert capacity指的是啥？为啥早期的架构需要drop tokens that exceed capacity?
会造成什么后果？目前的drop-less架构是如何做到的?

Why would a MoE have additional randomness?



Token dropping from routing happens at a *batch* level – this means that other people's queries can drop your token!

Issues with MoEs - stability



[7]Exponential functions have the property that a small input perturbation can lead to a large difference in the output. As an example, consider inputting 10 logits to a softmax function with values of 128 and one logit with a value 128.5. A roundoff error of 0.5 in `bfloat16` will alter the softmax output by 36% and incorrectly make all logits equal. The calculation goes from $\frac{\exp(0)}{\exp(0)+10\cdot\exp(-0.5)} \approx 0.142$ to $\frac{\exp(0)}{\exp(0)+10\cdot\exp(0)} \approx 0.091$. This occurs because the max is subtracted from all logits (for numerical stability) in softmax operations and the roundoff error changes the number from 128.5 to 128. This example was in `bfloat16`, but analogous situations occur in `float32` with larger logit values.

[Zoph 2022]

Solution: Use Float 32 just for the expert router (sometimes with an aux z-loss)

$$L_z(x) = \frac{1}{B}\sum_{i=1}^{B}\left(\log\sum_{j=1}^{N} e^{x_j^{(i)}}\right)^2 \tag{5}$$

#TODO：这里提到的问题和解决方式?

Z-loss stability for the router

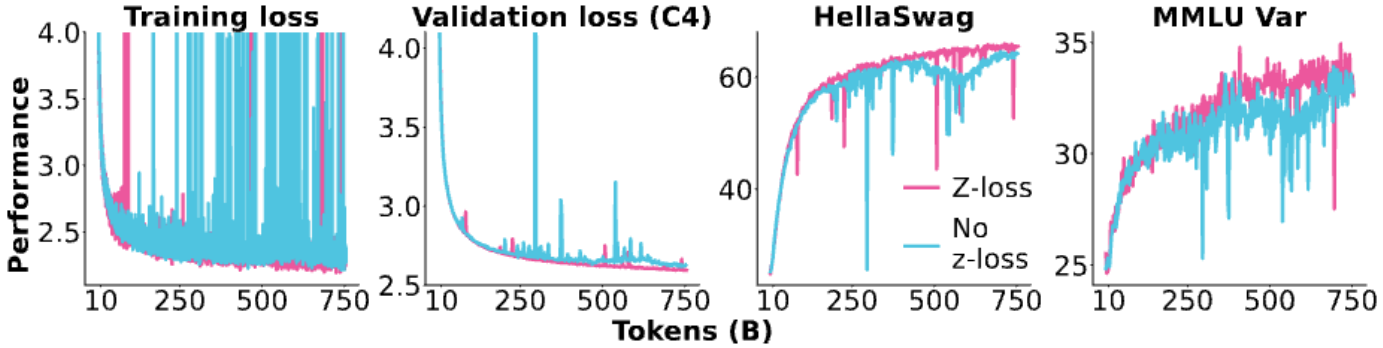


Figure 11: **Router z-loss.** We compare adding router z-loss with a loss weight of 0.001 versus no additional z-loss. More results, logs, and configurations: https://wandb.ai/ai2-llm/olmoe/reports/Plot-Zloss-vs-none--Vmlldzo4NDM4NjUz

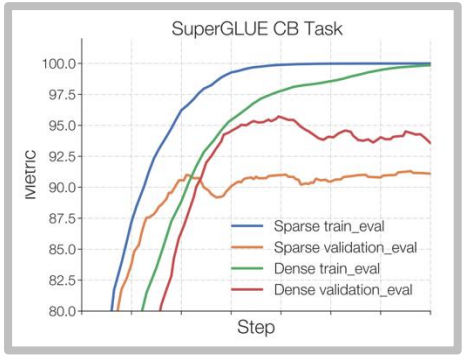
What happens when we remove the z-loss?

z-loss加入后，训练更加稳定了

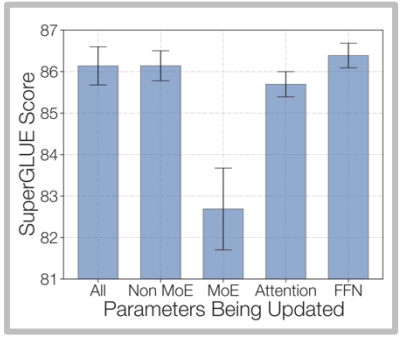
Issues with MoEs – fine-tuning

Sparse MoEs can overfit on smaller fine-tuning data

#TODO：这一页提到的问题和解决方式?



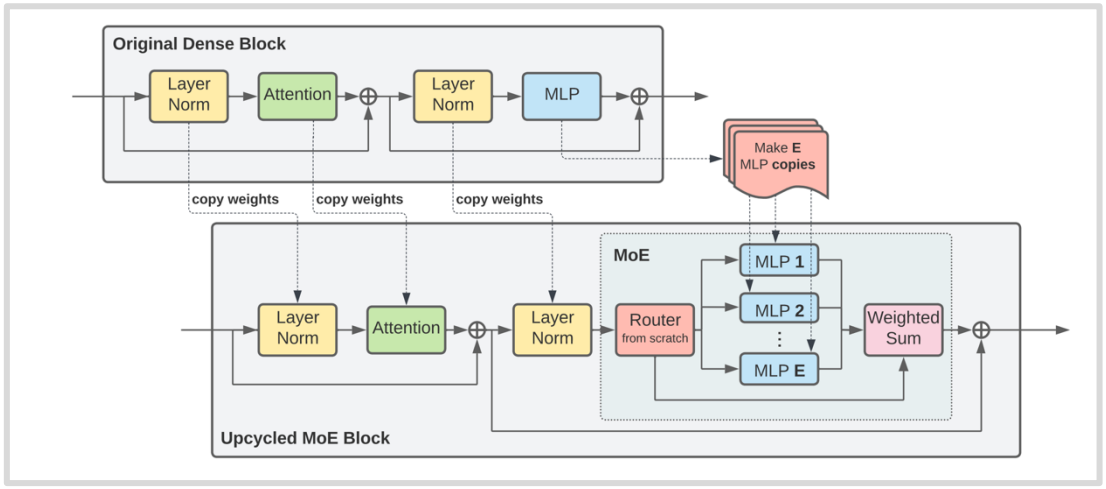
Zoph et al solution – finetune non-MoE MLPs

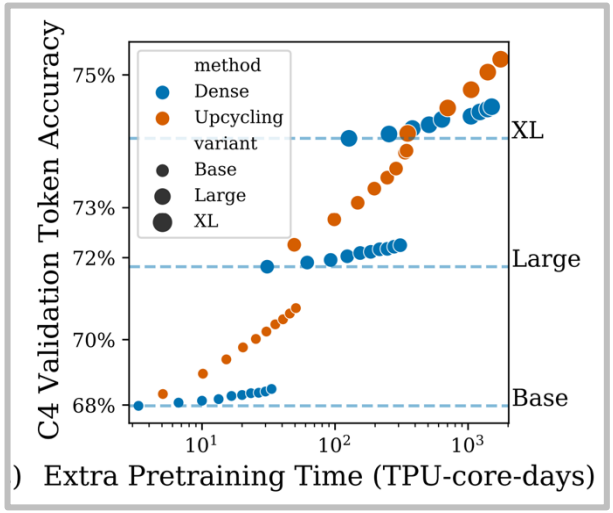


DeepSeek solution – use lots of data 1.4M SFT

Training Data. For training the chat model, we conduct supervised fine-tuning (SFT) on our in-house curated data, comprising 1.4M training examples. This dataset spans a broad range of categories including math, code, writing, question answering, reasoning, summarization, and more. The majority of our SFT training data is in English and Chinese, rendering the chat model versatile and applicable in bilingual scenarios.

Other training methods - upcycling





Can we use a pre-trained LM to initialize a MoE?

#TODO：这里的upcycling指的是啥？具体怎么做的？

Upcycling example - MiniCPM

Uses the MiniCPM model (topk=2, 8 experts, ~ 4B active params).

Model	C-Eval	CMMLU	MMLU	HumanEval	MBPP	GSM8K	MATH	BBH
Llama2-34B	-	-	62.6	22.6	33.0†	42.2	6.24	**44.1**
Deepseek-MoE (16B)	40.6	42.5	45.0	26.8	39.2	18.8	4.3	-
Mistral-7B	46.12	42.96	**62.69**	27.44	45.20	33.13	5.0	41.06
Gemma-7B	42.57	44.20	60.83	38.41	50.12	47.31	6.18	39.19
MiniCPM-2.4B	51.13	51.07	53.46	50.00	47.31	53.83	10.24	36.87
MiniCPM-MoE (13.6B)	**58.11**	**58.80**	58.90	**56.71**	**51.05**	**61.56**	**10.52**	39.22

Table 6: Benchmark results of MiniCPM-MoE. † means evaluation results on the full set of MBPP, instead of the hand-verified set (Austin et al., 2021). The evaluation results of Llama2-34B and Qwen1.5-7B are taken from their technical reports.

Simple MoE, shows gains from the base model with ~ 520B tokens for training

Upcycling example – Qwen MoE

Qwen MoE – Initialized from the Qwen 1.8B model top-k=4, 60 experts w/ 4 shared.

Model	#Parameters	#(Activated) Parameters	MMLU	GSM8K	HumanEval	Multilingual	MT-Bench
Mistral-7B	7.2	7.2	64.1	47.5	27.4	40.0	7.60
Qwen1.5-7B	7.7	7.7	64.6	50.9	32.3	-	-
Gemma-7B	8.5	7.8	61.0	62.5	36.0	45.2	7.60
DeepSeekMoE 16B	16.4	2.8	45.0	18.8	26.8	-	6.93
Qwen1.5-MoE-A2.7B	14.3	2.7	62.5	61.5	34.2	40.8	7.17

Similar architecture / setup to DeepSeekMoE, but one of the first (confirmed) upcycling successes

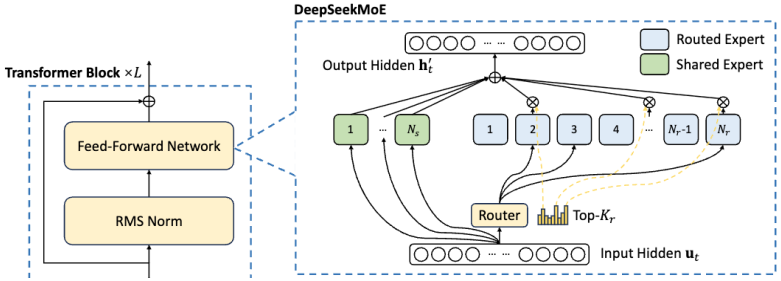
nowadays：直接训练一个MoE，而不是先训练dense，再将其转换为MoE。

DeepSeek MoE v1-v2-v3

To wrap up, we'll walk through the DeepSeek MoE architecture.

V1 (16B – 2.8 active):

Shared (2) + Fine-grained (64/4) experts



Standard, top-k routing

$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \mathrm{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t}\,\mathrm{FFN}_i^{(r)}(\mathbf{u}_t),$$

$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \mathrm{Topk}(\{s_{j,t} | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise}, \end{cases}$$

$$s_{i,t} = \mathrm{Softmax}_i\left(\mathbf{u}_t{}^T \mathbf{e}_i\right),$$

Standard Aux-loss balancing (Expert + Device)

$$\mathcal{L}_{\mathrm{ExpBal}} = \alpha_1 \sum_{i=1}^{N_r} f_i P_i,$$

$$f_i = \frac{N_r}{K_r T} \sum_{t=1}^{T} \mathbb{1}(\text{Token } t \text{ selects Expert } i),$$

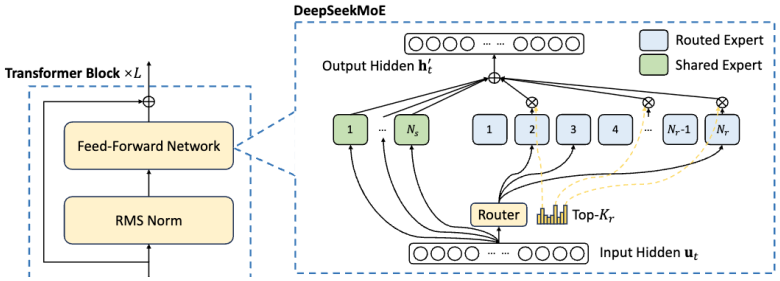
$$P_i = \frac{1}{T} \sum_{t=1}^{T} s_{i,t},$$

DeepSeek MoE v2

相比v1 scale up了

V2 (236B – 21 active):

Shared (2) + Fine-grained (160/10) experts, 6 active



New things:

Top-M device routing

For DeepSeek-V2, beyond the naive top-K selection of routed experts, we additionally ensure that the target experts of each token will be distributed on at most M devices. To be specific, for each token, we first select M devices that have experts with the highest affinity scores in them. Then, we perform top-K selection among experts on these M devices. In practice, we find that when $M \geqslant 3$, the device-limited routing can achieve a good performance roughly aligned with the unrestricted top-K routing.

结合system侧（这很重要），不能仅仅重视算法侧

Communication balancing loss – balancing both communication in *and* out

$$\mathcal{L}_{\text{CommBal}} = \alpha_3 \sum_{i=1}^{D} f''_i P''_i, \tag{29}$$

$$f''_i = \frac{D}{MT} \sum_{t=1}^{T} \mathbb{1}(\text{Token } t \text{ is sent to Device } i), \tag{30}$$

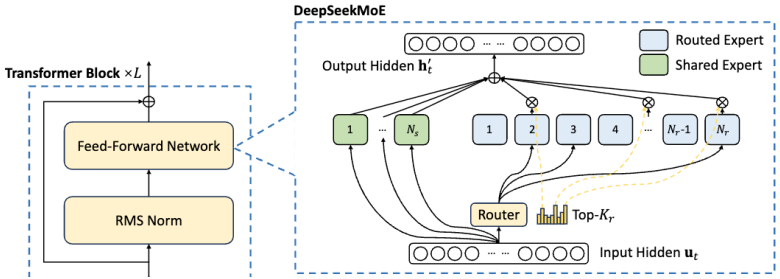
$$P''_i = \sum_{j \in \mathcal{E}_i} P_j, \tag{31}$$

where α_3 is a hyper-parameter called communication balance factor. The device-limited routing mechanism operates on the principle of ensuring that each device transmits at most MT hidden states to other devices. Simultaneously, the communication balance loss is employed to encourage each device to receive around MT hidden states from other devices. The communication balance loss guarantees a balanced exchange of information among devices, promoting efficient communications.

DeepSeek MoE v3

V2 (671B – 37 active):

Shared (1) + Fine-grained (258) experts, 8 active



New things

Sigmoid+Softmax topK + topM

$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \mathrm{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t}\,\mathrm{FFN}_i^{(r)}(\mathbf{u}_t),$$

$$g_{i,t} = \frac{g'_{i,t}}{\sum_{j=1}^{N_r} g'_{j,t}},$$

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \mathrm{Topk}(\{s_{j,t} | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise}, \end{cases}$$

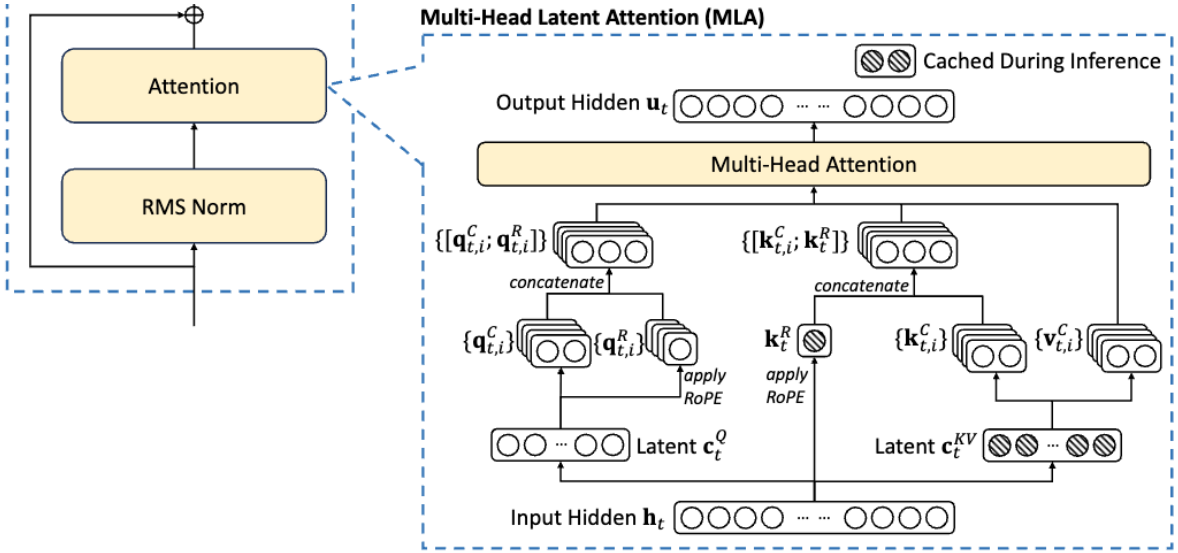
$$s_{i,t} = \mathrm{Sigmoid}\left(\mathbf{u}_t^T \mathbf{e}_i\right),$$

Aux-loss-free + seq-wise aux

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \mathrm{Topk}(\{s_{j,t} + b_j | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise}. \end{cases}$$

Bonus: What else do you need to make DeepSeek MoE v3?

MLA : Multihead, latent attention



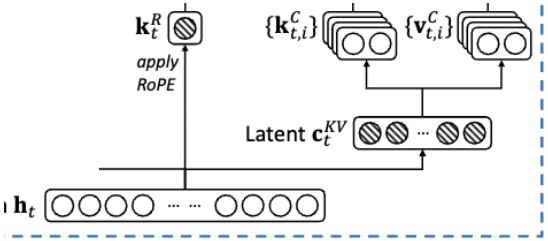
Basic idea: express the Q, K, V as functions of a lower-dim, 'latent' activation

#TODO：这里的latent c，具体是如何做的？具体是啥架构？为啥能节省KV cache?

What else do you need to make DeepSeek MoE v3?

Basic idea: express the Q, K, V as functions of a lower-dim, 'latent' activation

$$\mathbf{c}_t^{KV} = W^{DKV}\mathbf{h}_t,$$
$$\mathbf{k}_t^C = W^{UK}\mathbf{c}_t^{KV},$$
$$\mathbf{v}_t^C = W^{UV}\mathbf{c}_t^{KV},$$



Benefits: when KV-caching, we only need to store c_t^{KV}, which can be much smaller.
W^{UK} can be merged into the Q projection

(they also compress queries, for memory savings during training)

$$\mathbf{c}_t^Q = W^{DQ}\mathbf{h}_t,$$
$$\mathbf{q}_t^C = W^{UQ}\mathbf{c}_t^Q,$$

Complexity: rope conflicts with MLA-style caching.

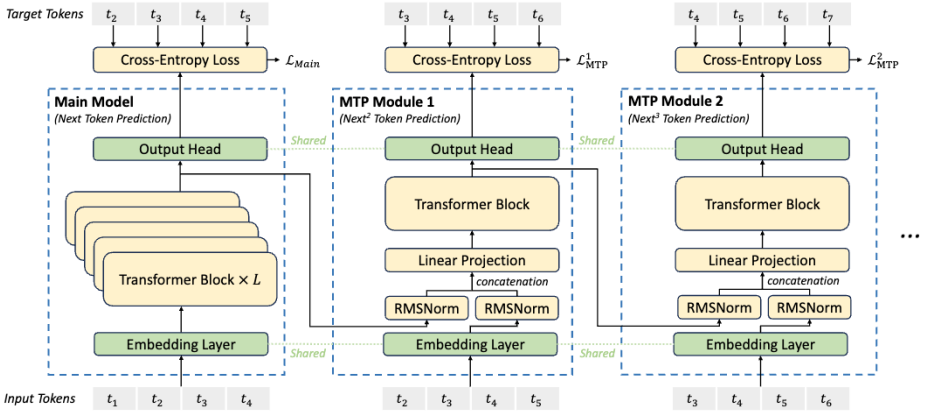
Without RoPE – $\langle Q, K\rangle = \langle hW^Q, W^{UK}c_t^{KV}\rangle = \langle h\, W^Q W^{UK}, c_t^{KV}\rangle$

With RoPE - $\langle QR_q, R_k K\rangle = \langle hW^Q R_q, R_k W^{UK}c_t^{KV}\rangle = \langle h\, W^Q R_q R_k W^{UK}, c_t^{KV}\rangle$

The solution – Have a few non-latent key dimensions that can be rotated

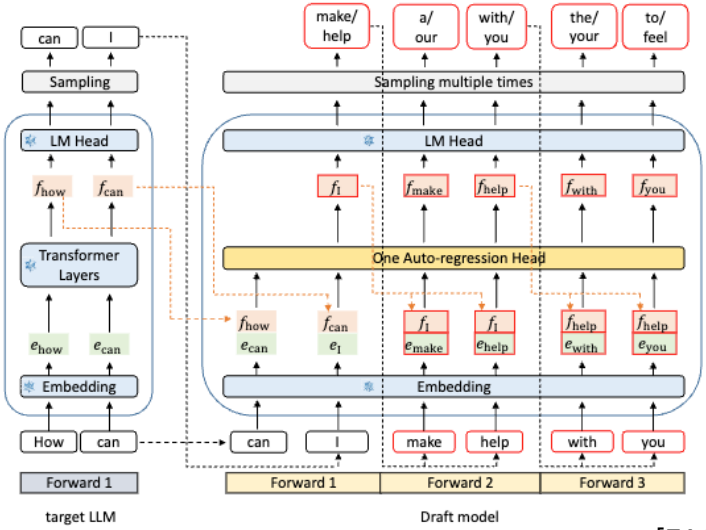
What else do you need to make DeepSeek MoE v3?

MTP: Have small, lightweight models that predict multiple steps ahead



(But they only do MTP with one token ahead)

[Deepseek v3]



[EAGLE]

$$\mathbf{h}_i'^k = M_k[\mathrm{RMSNorm}(\mathbf{h}_i^{k-1}); \mathrm{RMSNorm}(\mathrm{Emb}(t_{i+k}))],$$

$$\mathbf{h}_{1:T-k}^k = \mathrm{TRM}_k(\mathbf{h}_{1:T-k}'^k),$$

$$P_{i+k+1}^k = \mathrm{OutHead}(\mathbf{h}_i^k).$$

(See paper for ablations)

MoE summary

- MoEs take advantage of sparsity – not all inputs need the full model
  - Discrete routing is hard, but top-k heuristics seem to work
- Lots of empirical evidence now that MoEs work, and are cost-effective